```
#include <windows.h>
#include <d3d11.h>
#include <d3dcompiler.h>
#include <cmath> // ИСПРАВЛЕНИЕ: Подключили математические функции sqrt и pow для CPU

#include <commctrl.h>
#pragma comment(lib, "comctl32.lib")

#pragma comment(lib, "d3d11.lib")
#pragma comment(lib, "d3dcompiler.lib")

// === НОВЫЙ КОД: ИДЕНТИФИКАТОРЫ ДЛЯ ВЕРХНЕГО МЕНЮ WINDOWS ===
#define IDM_FILE_OPEN_VIDEO    1001  // Открыть видеофайл (AVI/MP4)
#define IDM_FILE_OPEN_IMAGE    1002  // Открыть отдельное изображение (BMP/PNG)
#define IDM_FILE_SAVE_RESULT   1003  // Записать преобразованный файл
#define IDM_FILE_EXIT          1004  // Выход из программы

#define IDM_VIEW_SPLIT         1005  // Режим "До / После" (разделение экрана)
#define IDM_VIEW_RESULT_ONLY   1006  // Режим "Только результат"

#define IDM_HELP_ABOUT         1007  // О программе
#define IDM_VIEW_CPU           40003
// Идентификаторы для дискретных масштабов дефектоскопа
#define IDM_ZOOM_FIT   40010
#define IDM_ZOOM_1     40011
#define IDM_ZOOM_2     40012
#define IDM_ZOOM_4     40013
#define IDM_ZOOM_8     40014
#define IDM_ZOOM_16    40015

// Структура вершины для нашего видеоэкрана
struct SimpleVertex
```

```
{
   float Pos[3]; // Координаты X, Y, Z в пространстве экрана
   float Tex[2]; // Текстурные координаты U, V для видеопотока
};

// Структура параметров кадра для передачи в шейдеры
struct ShaderConstants
{
   float width;    // Ширина картинки
   float height;   // Высота картинки
   float d_width;  // Шаг одного пикселя по горизонтали (1.0 / width)
   float d_height; // Шаг одного пикселя по вертикали (1.0 / height)
   float splitX;   // Положение шторки (доля от 0.0 до 1.0)
   float padding[3]; // Выравнивание структуры в памяти под требования DirectX (кратность 16 байт)
};

//
void CreateDiagnosticWindow(HINSTANCE hInstance, HWND hParentWnd);

// Прототип функции программной фильтрации на центральном процессоре
HRESULT ApplyCpuFilter(UINT* pSrcPixels, UINT width, UINT height, UINT* pOutPixels);

// Глобальный указатель на буфер констант
ID3D11Buffer* g_pConstantBuffer = NULL;

// ======================================================================
// ПОЛНЫЙ НАБОР СИСТЕМНЫХ ТЕКСТУР И СВЯЗЕЙ ДЛЯ ВСЕХ СТАДИЙ ДЕФЕКТОВЕДЕНИЯ
// ======================================================================

// СТАДИЯ 0: Исходное цветное изображение (Оригинал 1:1)
ID3D11Texture2D* g_pTexOriginal = nullptr;
ID3D11ShaderResourceView* g_pSRVOriginal = nullptr;
```

```
ID3D11Texture2D* g_pStagingOriginal = nullptr;

// СТАДИЯ 1: Монохромный кадр (Черно-белый оригинал 1:1)
ID3D11Texture2D* g_pTexMonochrome = nullptr;
ID3D11RenderTargetView* g_pRTVMonochrome = nullptr;
ID3D11ShaderResourceView* g_pSRVMonochrome = nullptr;
ID3D11Texture2D* g_pStagingMonochrome = nullptr;

// СТАДИЯ 2: Сжатое монохромное поле (Характеристика макроструктуры 4x4)
ID3D11Texture2D* g_pTexCompressed = nullptr;
ID3D11RenderTargetView* g_pRTVCompressed = nullptr;
ID3D11ShaderResourceView* g_pSRVCompressed = nullptr;
ID3D11Texture2D* g_pStagingCompressed = nullptr;

// ЭКСПЕРИМЕНТАЛЬНЫЙ ЭТАП: Дважды сжатое монохромное поле (Промежуточный масштаб 16x16)
ID3D11Texture2D* g_pTexDoubleCompressed = nullptr;
ID3D11RenderTargetView* g_pRTVDoubleCompressed = nullptr;
ID3D11ShaderResourceView* g_pSRVDoubleCompressed = nullptr;
ID3D11Texture2D* g_pStagingDoubleCompressed = nullptr;

// СТАДИЯ 3: Растянутое после сжатия (Глобальный сглаженный контраст без четких границ)
ID3D11Texture2D* g_pTexStretched = nullptr;
ID3D11RenderTargetView* g_pRTVStretched = nullptr;
ID3D11ShaderResourceView* g_pSRVStretched = nullptr;
ID3D11Texture2D* g_pStagingStretched = nullptr;

// СТАДИЯ 4: Разность (Исходное минус Растянутое = Выделенные четкие границы отдельно от контраста)
ID3D11Texture2D* g_pTexDifference = nullptr;
ID3D11RenderTargetView* g_pRTVDifference = nullptr;
ID3D11ShaderResourceView* g_pSRVDifference = nullptr;
ID3D11Texture2D* g_pStagingDifference = nullptr;
```

```
// СТАДИЯ 5: Преобразованное сжатое монохромное поле
ID3D11Texture2D* g_pTexTransformed = nullptr;
ID3D11RenderTargetView* g_pRTVTransformed = nullptr;
ID3D11ShaderResourceView* g_pSRVTransformed = nullptr;
ID3D11Texture2D* g_pStagingTransformed = nullptr;

// СТАДИЯ 6: Мастер-буфер для полноразмерной склейки шторки плеера "До / После"
ID3D11Texture2D* g_pTexMasterSplit = nullptr;
ID3D11RenderTargetView* g_pStageMasterRTV = nullptr;
ID3D11ShaderResourceView* g_pStageMasterSRV = nullptr;

// ОБЪЕКТЫ ДЛЯ ФИЛЬТРА ГЛОБАЛЬНОЙ КОНТРАСТНОСТИ (СТАРАЯ СИСТЕМНАЯ ЧАСТЬ 5-ГО ШЕЙДЕРА)
ID3D11RenderTargetView* g_pStage3RTV = nullptr;
ID3D11ShaderResourceView* g_pStage3SRV = nullptr;

// === ТЕКСТУРНЫЕ УКАЗАТЕЛИ ДЛЯ КОМПИЛЯЦИИ И ЗАПУСКА HLSL ШЕЙДЕРОВ ===
ID3D11VertexShader* g_pVertexShader = nullptr;
ID3D11VertexShader* g_pDiagnosticVS = nullptr;
ID3D11PixelShader* g_pPixelShader = nullptr;
ID3D11PixelShader* g_pPixelShaderCopy = nullptr;
ID3D11PixelShader* g_pPixelShaderDownsample = nullptr;
ID3D11PixelShader* g_pPixelShaderDown = nullptr;
ID3D11PixelShader* g_pPixelShaderDown4x4 = nullptr;
ID3D11PixelShader* g_pPixelShaderStage2 = nullptr;
ID3D11PixelShader* g_pPixelShaderStage3 = nullptr;
ID3D11PixelShader* g_pPixelShaderFinal = nullptr;
ID3D11PixelShader* g_pPixelShaderMonochrome = nullptr;

// СТАРЫЕ ИНДЕКСЫ МАСШТАБИРОВАНИЯ (ОСТАВЛЯЕМ ДЛЯ СОВМЕСТИМОСТИ С КОДОМ АНАТОЛИЯ ФЕДОРОВИЧА)
ID3D11RenderTargetView* g_pStageDown2RTV = NULL;
```

```
ID3D11ShaderResourceView* g_pStageDown2SRV = NULL;
ID3D11RenderTargetView* g_pStageDown4RTV = NULL;
ID3D11ShaderResourceView* g_pStageDown4SRV = NULL;
ID3D11RenderTargetView* g_pStageDown16RTV = NULL;
ID3D11ShaderResourceView* g_pStageDown16SRV = NULL;

// СТРОКИ, КОТОРЫЕ ИДУТ ДАЛЬШЕ И ОСТАЮТСЯ НЕИЗМЕННЫМИ:
HWND hwnd; // НЕ ИЗМЕНЯЕТСЯ (Страница 4 PDF)
HWND g_hStatusWnd = NULL;

// Размеры оригинального кадра
UINT g_srcImgWidth = 0;
UINT g_srcImgHeight = 0;

// Размеры кадра, дополненного до кратности 8
UINT g_paddedImgWidth = 0;
UINT g_paddedImgHeight = 0;

// Размеры сжатого кадра (Режим 3)
UINT g_compressedWidth = 0;
UINT g_compressedHeight = 0;

// === НАБОР ФЛАГОВ ДЛЯ ОДНОКРАТНОГО СКАЧИВАНИЯ КАДРА В CPU ===
bool g_bNeedCopyOriginal = true;    // Флаг для копии исходного цветного файла
bool g_bNeedCopyMonochrome = true;
bool g_bNeedCopyStage3 = true;      // Флаг для монохромного кадра (Стадия 3)
bool g_bNeedCopyCompressed = true;  // Флаг для макроструктуры 8x8 (Стадия 2)
bool g_bNeedCopyStretched = true;   // НОВЫЙ уникальный флаг для Режима 4!

// === ОБЪЯВЛЕНИЕ ВНЕШНИХ ПЕРЕМЕННЫХ ИЗ ПАНЕЛИ ДИАГНОСТИКИ ===
UINT* pPixelsBuffer = NULL;
UINT* pRawPixels = NULL;
```

```
UINT* cpu_pRawPixels = NULL;          // Исходный цветной кадр 1:1 для алгоритмов CPU
UINT* cpu_pMonochromeBuffer = NULL;    // Монохромный оригинал 1:1, рассчитанный процессором
UINT* cpu_pCompressed4x = NULL;       // Первый каскад сжатия CPU (макроструктура 4x4 / 8x8)
UINT* cpu_pDoubleCompressed16x = NULL; // ВТОРОЙ КАСКАД сжатия CPU (глобальный фон 16x16)
UINT* cpu_pStretchedBuffer = NULL;    // Растянутый процессором фон под гамму
UINT* cpu_pDifferenceBuffer = NULL;

// === ПОЛНЫЙ НАБОР ЭТАЛОННЫХ БУФЕРОВ ОЗУ СТРОГО ДЛЯ РАСЧЕТОВ CPU ===
UINT* pRawPixels = NULL;
UINT* pMonochromeBuffer = NULL;
UINT* pCompressedBuffer4x = NULL;  // Внешний буфер сжатия из my_debug.cpp
UINT* cpu_pDoubleCompressed16x = NULL;
UINT* pStretchedBuffer = NULL;    // Буфер для хранения кадра, растянутого обратно после сжатия
UINT* cpu_pDifferenceBuffer = NULL;
extern ID3D11Texture2D* g_pStagingCompressed;

#define my_DRBUG
#ifdef my_DEBUG
#include my_debug.c
#endif

// === НЕИЗМЕННЫЕ ПЕРЕМЕННЫЕ ПРОГРАММНОГО РЕЖИМА (CPU) ===
bool g_bUseCPUProcessing = false;     // true - процессор, false - видеокарта
ID3D11Texture2D* g_pCpuTexture = NULL;  // Промежуточный холст для пикселей от CPU
ID3D11ShaderResourceView* g_pCpuTextureSRV = NULL; // Ресурс чтения результата CPU для шторки

// === ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ДЛЯ ПОЛОС ПРОКРУТКИ ===
int                g_scrollX = 0; // Текущий сдвиг картинки по горизонтали в пикселях
int                g_scrollY = 0; // Текущий сдвиг картинки по вертикали в пикселях
UINT               g_currentImgWidth = 0;  // Храним размеры текущего файла
UINT               g_currentImgHeight = 0;
```

```
// === ГЛОБАЛЬНЫЕ РАЗМЕРЫ КЛИЕНТСКОЙ ОБЛАСТИ ОКНА ===
float		g_wndW = 800.0f; // Стартовая ширина окна по умолчанию
float		g_wndH = 600.0f; // Стартовая высота окна по умолчанию

// === ГЛОБАЛЬНЫЕ ДОЛИ ПРОКРУТКИ КАДРА (ОТ 0.0 ДО 1.0) ===
float		g_scrollRatioX = 0.0f;
float		g_scrollRatioY = 0.0f;

// Глобальный делитель масштаба кадра
float		g_zoomDivider = 1.0f;

// Глобальный флаг режима автоматического вписывания картинки в окно
bool		g_bFitToWindow = true; // По умолчанию плеер стартует в режиме "Вписать в кадр"!

// === ГЛОБАЛЬНЫЕ МАСШТАБНЫЕ МНОЖИТЕЛИ ДЛЯ ИДЕАЛЬНОГО СКРОЛЛИНГА ===
float		g_scrollMultiplierX = 1.0f;
float		g_scrollMultiplierY = 1.0f;

// === НОВЫЙ КОД: ПЕРЕМЕННАЯ ДЛЯ ХРАНЕНИЯ ПУТИ К ОТКРЫТОМУ ФАЙЛУ ===
wchar_t		g_szSelectedFilePath[MAX_PATH] = L""; // Буфер для пути к картинке или видео

// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (СТРОГО ПО ОДНОМУ ЭКЗЕМПЛЯРУ): ---
ID3D11Device* g_pd3dDevice = NULL;	// Видеокарта
ID3D11DeviceContext* g_pImmediateContext = NULL; // Контекст команд
IDXGISwapChain* g_pSwapChain = NULL;	// Буфер экрана
ID3D11RenderTargetView* g_pRenderTargetView = NULL; // Окно вывода

ID3D11InputLayout* g_pVertexLayout = NULL;	// Формат вершин
ID3D11Buffer* g_pVertexBuffer = NULL;	// Буфер геометрии
```

```
SimpleVertex g_currentVertices[4];    // Массив вершин, хранящий живую геометрию скроллинга окна


ID3D11ShaderResourceView* g_pTextureSRV = NULL;       // Наша текстура 2x2

ID3D11SamplerState* g_pSamplerState = NULL;     // Жесткий сэмплер (Point)


LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);


void CleanupDevice(); // НОВОЕ: Объявляем функцию очистки заранее, чтобы убрать ошибку компиляции


// Функция для компиляции HLSL-файлов

HRESULT CompileShaderFromFile(const WCHAR* szFileName, LPCSTR szEntryPoint, LPCSTR szShaderModel, ID3DBlob** ppBlobOut)

{

   HRESULT hr = S_OK;

   ID3DBlob* pErrorBlob = NULL;


   hr = D3DCompileFromFile(szFileName, NULL, NULL, szEntryPoint, szShaderModel,

      D3DCOMPILE_ENABLE_STRICTNESS, 0, ppBlobOut, &pErrorBlob);


   // === ИСПРАВЛЕНИЕ: ТЕПЕРЬ ПЕРЕХВАТ НА СВОЕМ ЗАКОННОМ МЕСТЕ ВНУТРИ ФУНКЦИИ ===

   if (FAILED(hr))

   {

      if (pErrorBlob != NULL)

      {

         char* compileErrors = (char*)(pErrorBlob->GetBufferPointer());

         size_t newsize = strlen(compileErrors) + 1;

         wchar_t* wcstring = new wchar_t[newsize];

         size_t convertedChars = 0;

         mbstowcs_s(&convertedChars, wcstring, newsize, compileErrors, _TRUNCATE);


         // Показываем на экране точную строчную ошибку из файла shaders.hlsl
```

```
        MessageBoxW(NULL, wcstring, L"Критическая ошибка компиляции HLSL!", MB_OK | MB_ICONERROR);

        delete[] wcstring;
        pErrorBlob->Release();
    }
    else
    {
        MessageBoxW(NULL, L"Файл shaders.hlsl не найден или поврежден!", L"Ошибка", MB_OK | MB_ICONERROR);
    }
    return hr;
  }

  return S_OK;
}

// Инициализация видеокарты и создание объектов
HRESULT InitDevice(HWND hwnd)
{
  HRESULT hr = S_OK;

  RECT rc;
  GetClientRect(hwnd, &rc);
  UINT width = rc.right - rc.left;
  UINT height = rc.bottom - rc.top;

  DXGI_SWAP_CHAIN_DESC sd = {};
  sd.BufferCount = 1;
  sd.BufferDesc.Width = width;
  sd.BufferDesc.Height = height;
  sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
  sd.BufferDesc.RefreshRate.Numerator = 60;
```

```
sd.BufferDesc.RefreshRate.Denominator = 1;
sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
sd.OutputWindow = hwnd;
sd.SampleDesc.Count = 1;
sd.SampleDesc.Quality = 0;
sd.Windowed = TRUE;

D3D_DRIVER_TYPE driverType = D3D_DRIVER_TYPE_HARDWARE;
D3D_FEATURE_LEVEL featureLevel = D3D_FEATURE_LEVEL_11_0;

hr = D3D11CreateDeviceAndSwapChain(NULL, driverType, NULL, 0, &featureLevel, 1,
   D3D11_SDK_VERSION, &sd, &g_pSwapChain, &g_pd3dDevice, NULL, &g_pImmediateContext);
if (FAILED(hr)) return hr;

ID3D11Texture2D* pBackBuffer = NULL;
hr = g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
pBackBuffer->Release();
if (FAILED(hr)) return hr;

g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

D3D11_VIEWPORT vp;
vp.Width = (FLOAT)width;
vp.Height = (FLOAT)height;
vp.MinDepth = 0.0f;
vp.MaxDepth = 1.0f;
vp.TopLeftX = 0;
vp.TopLeftY = 0;
g_pImmediateContext->RSSetViewports(1, &vp);
```

```
    // Компиляция Вершинного Шейдера
    ID3DBlob* pVSBlob = NULL;
    hr = CompileShaderFromFile(L"shaders.hlsl", "VS", "vs_4_0", &pVSBlob);
    // === ИСПРАВЛЕНИЕ: ПЕРЕХВАТ ТЕКСТА ОШИБКИ ИЗ SHADERS.HLSL ===
    if (FAILED(hr)) return hr; // Вернули стандартную проверку, ошибка C2065 исчезнет!


    hr = g_pd3dDevice->CreateVertexShader(pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(), NULL, &g_pVertexShader);


    if (FAILED(hr)) {
        MessageBoxW(hwnd, L"Ошибка на Шаге Ш-VS: Видеокарта отвергла Вершинный Шейдер!", L"Сбой дефектоскопа", MB_OK | MB_ICONERROR);
        pVSBlob->Release();
        return hr;
    }


    // Описываем формат вершин для видеокарты (Input Layout)
    D3D11_INPUT_ELEMENT_DESC layout[] =
    {
        { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
        { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
    };
    UINT numElements = sizeof(layout) / sizeof(layout[0]);


    hr = g_pd3dDevice->CreateInputLayout(layout, numElements, pVSBlob->GetBufferPointer(), pVSBlob->GetBufferSize(), &g_pVertexLayout);
    pVSBlob->Release();
    if (FAILED(hr)) return hr;


    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);
```

```
    // Компиляция Пиксельного Шейдера
    ID3DBlob* pPSBlob = NULL;
    hr = CompileShaderFromFile(L"shaders.hlsl", "PS", "ps_4_0", &pPSBlob);
    if (FAILED(hr)) return hr;

    hr = g_pd3dDevice->CreatePixelShader(pPSBlob->GetBufferPointer(), pPSBlob->GetBufferSize(), NULL,
&g_pPixelShader);
    pPSBlob->Release();
    if (FAILED(hr)) return hr;

    // КОМПИЛЯЦИЯ И СОЗДАНИЕ НАШЕГО ШЕЙДЕРА КОПИРОВАНИЯ
    ID3DBlob* pPSCopyBlob = NULL;
    hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Copy", "ps_4_0", &pPSCopyBlob);
    if (FAILED(hr)) return hr;
    hr = g_pd3dDevice->CreatePixelShader(pPSCopyBlob->GetBufferPointer(), pPSCopyBlob-
>GetBufferSize(), NULL, &g_pPixelShaderCopy);
    pPSCopyBlob->Release();
    if (FAILED(hr)) return hr;

    // === КОМПИЛЯЦИЯ НОВОГО ШЕЙДЕРА ПИРАМИДАЛЬНОГО СЖАТИЯ ===
    ID3DBlob* pDownsampleBlob = NULL; // ОБЪЯВЛЯЕМ ПРОПУЩЕННЫЙ БУФЕР ДЛЯ СЖАТИЯ
    hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Downsample2X", "ps_4_0", &pDownsampleBlob);
    if (FAILED(hr)) return hr;

    hr = g_pd3dDevice->CreatePixelShader(pDownsampleBlob->GetBufferPointer(), pDownsampleBlob-
>GetBufferSize(), NULL, &g_pPixelShaderDownsample);
    pDownsampleBlob->Release(); // Чисто освобождаем буфер
    if (FAILED(hr)) return hr;

    // === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 4-ГО ШЕЙДЕРА (STAGE 2) ===
    ID3DBlob* pPSStage2Blob = NULL;
    // Указываем точку входа "PS_Stage2" — строго как имя функции в файле shaders.hlsl
    hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage2", "ps_4_0", &pPSStage2Blob);
```

```
    if (FAILED(hr)) return hr;


    hr = g_pd3dDevice->CreatePixelShader(pPSStage2Blob->GetBufferPointer(), pPSStage2Blob->GetBufferSize(), NULL, &g_pPixelShaderStage2);
    pPSStage2Blob->Release(); // Освобождаем память временного буфера компиляции
    if (FAILED(hr)) return hr;
    // === КОНЕЦ НОВОГО КОДА ===


    ID3DBlob* pVSDiagBlob = NULL;
    hr = CompileShaderFromFile(L"shaders.hlsl", "VS_DiagnosticQuad", "vs_4_0", &pVSDiagBlob);
    if (SUCCEEDED(hr))
    {
        g_pd3dDevice->CreateVertexShader(pVSDiagBlob->GetBufferPointer(), pVSDiagBlob->GetBufferSize(), NULL, &g_pDiagnosticVS);
        pVSDiagBlob->Release();
    }


    // === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 5-ГО ШЕЙДЕРА (STAGE 3) ===
    ID3DBlob* pPSStage3Blob = NULL;
    // Указываем точку входа "PS_Stage3" — строго как имя функции в файле shaders.hlsl
    hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Stage3", "ps_4_0", &pPSStage3Blob);
    if (FAILED(hr)) return hr;


    hr = g_pd3dDevice->CreatePixelShader(pPSStage3Blob->GetBufferPointer(), pPSStage3Blob->GetBufferSize(), NULL, &g_pPixelShaderStage3);
    pPSStage3Blob->Release(); // Освобождаем память временного буфера компиляции
    if (FAILED(hr)) return hr;
    // === КОНЕЦ НОВОГО КОДА ===


    // === КОМПИЛЯЦИЯ И СОЗДАНИЕ ШЕЙДЕРА СЖАТИЯ 4x4 ===
    ID3DBlob* pPSBlobDown = nullptr;
    ID3DBlob* pErrorBlobLocal = nullptr;
```

```
  hr = D3DCompileFromFile(L"shaders.hlsl", nullptr, nullptr, "PS_Downsample4x4", "ps_5_0", D3DCOMPILE_ENABLE_STRICTNESS, 0, &pPSBlobDown, &pErrorBlobLocal);
  if (FAILED(hr))
  {
    if (pErrorBlobLocal)
    {
      MessageBoxA(hwnd, (char*)pErrorBlobLocal->GetBufferPointer(), "Ошибка HLSL", MB_OK | MB_ICONERROR);
      pErrorBlobLocal->Release();
    }
    return hr;
  }

  // === КОМПИЛЯЦИЯ И СОЗДАНИЕ ШЕЙДЕРА СЖАТИЯ 8x8 ===
  ID3DBlob* pPSBlobDown8 = nullptr;
  ID3DBlob* pErrorBlobLocal8 = nullptr;

  hr = D3DCompileFromFile(L"shaders.hlsl", nullptr, nullptr, "PS_Downsample4x4", "ps_5_0", D3DCOMPILE_ENABLE_STRICTNESS, 0, &pPSBlobDown8, &pErrorBlobLocal8);
  if (FAILED(hr))
  {
    if (pErrorBlobLocal8) pErrorBlobLocal8->Release();
    MessageBox(hwnd, L"Не удалось скомпилировать шейдер PS_Downsample4x4!", L"Ошибка", MB_OK | MB_ICONERROR);
    return hr;
  }

  hr = g_pd3dDevice->CreatePixelShader(pPSBlobDown8->GetBufferPointer(), pPSBlobDown8->GetBufferSize(), nullptr, &g_pPixelShaderDown4x4);
  pPSBlobDown8->Release();
  if (FAILED(hr))
  {
    MessageBox(hwnd, L"Не удалось создать пиксельный шейдер сжатия 8x8!", L"Ошибка", MB_OK | MB_ICONERROR);
```

```
    return hr;
  }


  hr = g_pd3dDevice->CreatePixelShader(pPSBlobDown->GetBufferPointer(), pPSBlobDown->GetBufferSize(), nullptr, &g_pPixelShaderDown);
  pPSBlobDown->Release();
  if (FAILED(hr))
  {
    if (pErrorBlobLocal)
    {
      OutputDebugStringA((char*)pErrorBlobLocal->GetBufferPointer());
      MessageBoxA(hwnd, (char*)pErrorBlobLocal->GetBufferPointer(), "Ошибка HLSL", MB_OK | MB_ICONERROR);
      pErrorBlobLocal->Release();
    }
    return hr;
  }

  // === НОВЫЙ КОД: КОМПИЛЯЦИЯ И СОЗДАНИЕ 6-ГО ШЕЙДЕРА (FINAL) ===
  ID3DBlob* pPSFinalBlob = NULL;
  // Указываем точку входа "PS_Final" — строго как имя функции в файле shaders.hlsl
  hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Final", "ps_4_0", &pPSFinalBlob);
  if (FAILED(hr)) return hr;

  hr = g_pd3dDevice->CreatePixelShader(pPSFinalBlob->GetBufferPointer(), pPSFinalBlob->GetBufferSize(), NULL, &g_pPixelShaderFinal);
  pPSFinalBlob->Release(); // Освобождаем память временного буфера компиляции
  if (FAILED(hr)) return hr;

  ID3DBlob* pPSMonoBlob = NULL;
  hr = CompileShaderFromFile(L"shaders.hlsl", "PS_Monochrome", "ps_4_0", &pPSMonoBlob);
  if (FAILED(hr)) return hr;
```

```
    hr = g_pd3dDevice->CreatePixelShader(pPSMonoBlob->GetBufferPointer(), pPSMonoBlob->GetBufferSize(), NULL, &g_pPixelShaderMonochrome);
    pPSMonoBlob->Release();
    if (FAILED(hr)) return hr;
    // === КОНЕЦ НОВОГО КОДА ===

    // Создаем тестовую текстуру 2x2 пикселя (Черный и Белый в шахматном порядке)
    UINT textureData[] = {
        0xFFFFFFFF, 0xFF000000,
        0xFF000000, 0xFFFFFFFF
    };

    D3D11_TEXTURE2D_DESC desc = {};
    desc.Width = 2;
    desc.Height = 2;
    desc.MipLevels = 1;
    desc.ArraySize = 1;
    desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    desc.SampleDesc.Count = 1;
    desc.Usage = D3D11_USAGE_DEFAULT;
    desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

    D3D11_SUBRESOURCE_DATA initData = {};
    initData.pSysMem = textureData;
    initData.SysMemPitch = 2 * sizeof(UINT);

    ID3D11Texture2D* pTexture = NULL;
    hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);
    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);
        pTexture->Release();
    }
```

```
    if (FAILED(hr)) return hr;

    // Настраиваем режим POINT-фильтрации (строго без размытия)
    D3D11_SAMPLER_DESC sampDesc = {};
    sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;
    sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;
    sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;
    sampDesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;
    sampDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;
    sampDesc.MinLOD = 0;
    sampDesc.MaxLOD = D3D11_FLOAT32_MAX;

    hr = g_pd3dDevice->CreateSamplerState(&sampDesc, &g_pSamplerState);
    if (FAILED(hr)) return hr;

    // === НОВЫЙ КОД: СОЗДАЕМ БУФЕР КОНСТАНТ НА ВИДЕОКАРТЕ ===
    D3D11_BUFFER_DESC cbd = {};
    cbd.Usage = D3D11_USAGE_DEFAULT;
    cbd.ByteWidth = sizeof(ShaderConstants);      // Размер нашей структуры параметров
    cbd.BindFlags = D3D11_BIND_CONSTANT_BUFFER;  // Указываем видеокарте, что это буфер
константант
    cbd.CPUAccessFlags = 0;

    hr = g_pd3dDevice->CreateBuffer(&cbd, NULL, &g_pConstantBuffer);
    if (FAILED(hr)) return hr;
    // === КОНЕЦ НОВОГО КОДА ===

     // === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ АППАРАТНОГО МОНОХРОМА (СТАДИЯ 1)
===
    D3D11_TEXTURE2D_DESC monoDesc = {};
    monoDesc.Width = 2;
    monoDesc.Height = 2;
    monoDesc.MipLevels = 1;
```

```
    monoDesc.ArraySize = 1;
    monoDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    monoDesc.SampleDesc.Count = 1;
    monoDesc.Usage = D3D11_USAGE_DEFAULT;
    monoDesc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
    ID3D11Texture2D* pMonoTex = NULL;
    hr = g_pd3dDevice->CreateTexture2D(&monoDesc, NULL, &pMonoTex);
    if (SUCCEEDED(hr))
    {
        g_pd3dDevice->CreateRenderTargetView(pMonoTex, NULL, &g_pRTVMonochrome);
        g_pd3dDevice->CreateShaderResourceView(pMonoTex, NULL, &g_pSRVMonochrome);
        pMonoTex->Release();
    }
    if (FAILED(hr)) return hr;

    // === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 3-ГО ШЕЙДЕРА ===
    D3D11_TEXTURE2D_DESC stage1Desc = {};
    stage1Desc.Width = 2;                       // Размер точно соответствует исходному кадру
    stage1Desc.Height = 2;
    stage1Desc.MipLevels = 1;
    stage1Desc.ArraySize = 1;
    stage1Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;   // Стандартный RGBA формат для хранения данных
    stage1Desc.SampleDesc.Count = 1;
    stage1Desc.Usage = D3D11_USAGE_DEFAULT;
    // Очень важные флаги: текстура будет одновременно холстом для записи и ресурсом для чтения!
    stage1Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

    ID3D11Texture2D* pStage1Tex = NULL;
    hr = g_pd3dDevice->CreateTexture2D(&stage1Desc, NULL, &pStage1Tex);
    if (SUCCEEDED(hr))
    {
```

```
    // Создаем "взгляд для записи" (RenderTargetView) — сюда 3-й шейдер будет писать данные
    g_pd3dDevice->CreateRenderTargetView(pStage1Tex, NULL, &g_pRTVCompressed);

    // Создаем "взгляд для чтения" (ShaderResourceView) — отсюда 4-й шейдер будет их забирать
    g_pd3dDevice->CreateShaderResourceView(pStage1Tex, NULL, &g_pSRVCompressed);

    pStage1Tex->Release(); // Саму текстуру видеокарта удержит внутри интерфейсов
  }
  if (FAILED(hr)) return hr;
  // === КОНЕЦ НОВОГО КОДА ===

    // === НОВЫЙ КОД: СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 4-ГО ШЕЙДЕРА ===
  D3D11_TEXTURE2D_DESC stage2Desc = {};
  stage2Desc.Width = 2;                    // Размер по-прежнему соответствует тестовому кадру
  stage2Desc.Height = 2;
  stage2Desc.MipLevels = 1;
  stage2Desc.ArraySize = 1;
  stage2Desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;   // Формат RGBA для хранения
макростатистики
  stage2Desc.SampleDesc.Count = 1;
  stage2Desc.Usage = D3D11_USAGE_DEFAULT;
  stage2Desc.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

  ID3D11Texture2D* pStage2Tex = NULL;
  hr = g_pd3dDevice->CreateTexture2D(&stage2Desc, NULL, &pStage2Tex);
  if (SUCCEEDED(hr))
  {
    // Создаем RenderTargetView — сюда 4-й шейдер запишет укрупненные данные 8x8
    g_pd3dDevice->CreateRenderTargetView(pStage2Tex, NULL, &g_pRTVStretched);

    // Создаем ShaderResourceView — отсюда данные заберет финальный сборочный шейдер
    g_pd3dDevice->CreateShaderResourceView(pStage2Tex, NULL, &g_pSRVStretched);
```

```
    pStage2Tex->Release();
}
if (FAILED(hr)) return hr;
// === КОНЕЦ НОВОГО КОДА ===

ID3D11Texture2D* pStage3Tex = NULL; // НОВОЕ: Объявляем указатель для третьей текстуры
// СОЗДАЕМ СКРЫТУЮ ТЕКСТУРУ ДЛЯ РЕЗУЛЬТАТОВ 5-ГО ШЕЙДЕРА
D3D11_TEXTURE2D_DESC stage3Desc = stage2Desc; // Копируем настройки формата и размеров
hr = g_pd3dDevice->CreateTexture2D(&stage3Desc, NULL, &pStage3Tex);

if (SUCCEEDED(hr))
{
    // Создаем RenderTargetView — сюда 5-й шейдер запишет результат фильтрации контраста
    hr = g_pd3dDevice->CreateRenderTargetView(pStage3Tex, NULL, &g_pStage3RTV);

    // Создаем ShaderResourceView — отсюда данные пойдут на финальный экран
    if (SUCCEEDED(hr))
    {
        hr = g_pd3dDevice->CreateShaderResourceView(pStage3Tex, NULL, &g_pStage3SRV);
    }

    pStage3Tex->Release(); // Передаем управление текстурой видеокарте
}
if (FAILED(hr)) return hr;

// Геометрия прямоугольного экрана по умолчанию (стартовая)
SimpleVertex vertices[] =
{
{ { -1.0f,  1.0f, 0.0f },{ 0.0f, 0.0f } },
{ { 1.0f,  1.0f, 0.0f },{ 1.0f, 0.0f } },
{ { -1.0f, -1.0f, 0.0f },{ 0.0f, 1.0f } },
{ { 1.0f, -1.0f, 0.0f },{ 1.0f, 1.0f } },
```

```
    };

    D3D11_BUFFER_DESC bd = {};
    bd.Usage = D3D11_USAGE_DEFAULT;
    bd.ByteWidth = sizeof(SimpleVertex) * 4;
    bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
    bd.CPUAccessFlags = 0;

    D3D11_SUBRESOURCE_DATA InitData = {};
    InitData.pSysMem = vertices;

    hr = g_pd3dDevice->CreateBuffer(&bd, &InitData, &g_pVertexBuffer);
    if (FAILED(hr)) return hr;

    return S_OK;
}

// Обновленная функция отрисовки (Двухпроходный конвейер рендеринга)
// Обновленная функция отрисовки (Трехпроходный вычислительный конвейер)
// Обновленная функция отрисовки (Четырехпроходный вычислительный конвейер)
// Финальная функция отрисовки (Пятипроходный вычислительный конвейер)
// ПОЛНОЦЕННАЯ СИНХРОНИЗИРОВАННАЯ ФУНКЦИЯ ОТРИСОВКИ (ПЯТИПРОХОДНЫЙ КОНВЕЙЕР)
// ПРЯМОЙ СКВОЗНОЙ КОНВЕЙЕР: ИСКЛЮЧАЕТ ЛЮБЫЕ БЛОКИРОВКИ ТЕКСТУР В ПАМЯТИ
// ПОЛНОЦЕННАЯ СИНХРОНИЗИРОВАННАЯ ФУНКЦИЯ ОТРИСОВКИ С АВТОМАТИЧЕСКИМ СКРОЛЛИНГОМ И СОХРАНЕНИЕМ ПРОПОРЦИЙ
// УТРЕННЯЯ ИСПРАВЛЕННАЯ ФУНКЦИЯ ОТРИСОВКИ С УМНЫМ ПОРТОМ ПРОСМОТРА
// ТОЧНАЯ КОПИЯ УТРЕННЕЙ РАБОЧЕЙ ФУНКЦИИ ОТРИСОВКИ (МЯГКИЕ ПОЛУТОНА И СЕРОЕ ПОЛЕ)
void Render()
{
    // ЖЕСТКИЙ ЗАМОК БЕЗОПАСНОСТИ: Не пускаем видеокарту рендерить, пока не загружен файл
    if (g_pRenderTargetView == NULL || g_pTextureSRV == NULL || g_pStageMasterRTV == NULL) return;

    // 1. Настройка общих параметров сетки вершин
```

```
    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;
    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);

    // Привязываем базовые шейдеры и постоянные ресурсы конвейера
    g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
    g_pImmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
    g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);

    // Локальные переменные и цвета очистки буферов
    ID3D11ShaderResourceView* nullSRV[] = { NULL };
    ID3D11RenderTargetView* nullRTV[] = { NULL };
    float ClearColorBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };
    float ClearColorGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };

    // Автоматически и безопасно получаем точные размеры загруженного файла напрямую из текстуры
    UINT fileWidth = 32, fileHeight = 32;
    if (g_pTextureSRV) {
        ID3D11Resource* pRes = NULL;
        g_pTextureSRV->GetResource(&pRes);
        if (pRes) {
            ID3D11Texture2D* pTex2D = (ID3D11Texture2D*)pRes;
            D3D11_TEXTURE2D_DESC tDesc;
            pTex2D->GetDesc(&tDesc);
            fileWidth = tDesc.Width;
            fileHeight = tDesc.Height;
            pRes->Release();
        }
    }
    float renderW = (float)fileWidth;
    float renderH = (float)fileHeight;
```

```
  // === КОНТРОЛЬ GPU: ОДНОКРАТНОЕ СКАЧИВАНИЕ ИСХОДНОГО ОРИГИНАЛА ===
  if (g_bNeedCopyOriginal && g_pTexOriginal && g_pStagingOriginal && pRawPixels != NULL)
  {
    g_pImmediateContext->CopyResource(g_pStagingOriginal, g_pTexOriginal);
    D3D11_MAPPED_SUBRESOURCE mappedResource;
    if (SUCCEEDED(g_pImmediateContext->Map(g_pStagingOriginal, 0, D3D11_MAP_READ, 0,
&mappedResource)))
    {
      BYTE* pSrc = (BYTE*)mappedResource.pData;
      BYTE* pDst = (BYTE*)pRawPixels;
      UINT rowSize = g_currentImgWidth * sizeof(UINT);
      for (UINT y = 0; y < g_currentImgHeight; y++)
      {
        memcpy(pDst + (y * rowSize), pSrc + (y * mappedResource.RowPitch), rowSize);
      }
      g_pImmediateContext->Unmap(g_pStagingOriginal, 0);
    }
    g_bNeedCopyOriginal = false;
  }

  // === ДИНАМИЧЕСКОЕ ПЕРЕСОЗДАНИЕ МОНОХРОМНОГО ХОЛСТА ПОД РАЗМЕР КАДРА ===
  if (g_pTexMonochrome)
  {
    D3D11_TEXTURE2D_DESC mDesc;
    g_pTexMonochrome->GetDesc(&mDesc);
    if (mDesc.Width != fileWidth || mDesc.Height != fileHeight)
    {
      if (g_pSRVMonochrome) { g_pSRVMonochrome->Release(); g_pSRVMonochrome = nullptr; }
      if (g_pRTVMonochrome) { g_pRTVMonochrome->Release(); g_pRTVMonochrome = nullptr; }
      g_pTexMonochrome->Release();
      g_pTexMonochrome = nullptr;
    }
```

```
    }
    if (g_pTexMonochrome == nullptr)
    {
        D3D11_TEXTURE2D_DESC descMono = {};
        descMono.Width = fileWidth;
        descMono.Height = fileHeight;
        descMono.MipLevels = 1;
        descMono.ArraySize = 1;
        descMono.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
        descMono.SampleDesc.Count = 1;
        descMono.Usage = D3D11_USAGE_DEFAULT;
        descMono.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
        if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descMono, nullptr, &g_pTexMonochrome)))
        {
            g_pd3dDevice->CreateRenderTargetView(g_pTexMonochrome, nullptr, &g_pRTVMonochrome);
            g_pd3dDevice->CreateShaderResourceView(g_pTexMonochrome, nullptr,
&g_pSRVMonochrome);
        }
    }

    // Вычисляем размеры для Стадии 1 (строго в 4 раза меньше оригинала)
    UINT s1Width = (fileWidth / 4 > 0) ? fileWidth / 4 : 1;
    UINT s1Height = (fileHeight / 4 > 0) ? fileHeight / 4 : 1;

    // Если холст уже существует, но его размеры не совпадают с новыми — пересоздаем его
    if (g_pTexCompressed)
    {
        D3D11_TEXTURE2D_DESC currentDesc;
        g_pTexCompressed->GetDesc(&currentDesc);
        if (currentDesc.Width != s1Width || currentDesc.Height != s1Height)
        {
            if (g_pStageDown4SRV) { g_pStageDown4SRV->Release(); g_pStageDown4SRV = nullptr; }
            if (g_pStageDown4RTV) { g_pStageDown4RTV->Release(); g_pStageDown4RTV = nullptr; }
```

```
        g_pTexCompressed->Release();
        g_pTexCompressed = nullptr;
    }
}

// Физически выделяем память на видеокарте, если холста еще нет
if (g_pTexCompressed == nullptr)
{
    D3D11_TEXTURE2D_DESC descS1 = {};
    descS1.Width = s1Width;
    descS1.Height = s1Height;
    descS1.MipLevels = 1;
    descS1.ArraySize = 1;
    descS1.Format = DXGI_FORMAT_R32G32B32A32_FLOAT; // 4 канала для характеристик 2011 года
    descS1.SampleDesc.Count = 1;
    descS1.SampleDesc.Quality = 0;
    descS1.Usage = D3D11_USAGE_DEFAULT;
    descS1.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
    descS1.CPUAccessFlags = 0;
    descS1.MiscFlags = 0;

    HRESULT hr_s1 = g_pd3dDevice->CreateTexture2D(&descS1, nullptr, &g_pTexCompressed);
    if (SUCCEEDED(hr_s1))
    {
        // Привязываем текстуру Стадии 1 к её СОБСТВЕННЫМ уникальным указателям
        g_pd3dDevice->CreateRenderTargetView(g_pTexCompressed, nullptr, &g_pRTVCompressed);
        g_pd3dDevice->CreateShaderResourceView(g_pTexCompressed, nullptr, &g_pSRVCompressed);
    }
}

// === НАСТРОЙКА ОБЛАСТИ ПРОСМОТРА ДЛЯ СТАДИИ 1 (СЖАТИЕ 4x4) ===
D3D11_VIEWPORT vpStage1 = {};
```

```
    vpStage1.TopLeftX = 0.0f;
    vpStage1.TopLeftY = 0.0f;
    vpStage1.Width = (float)s1Width;   // Ширина в 4 раза меньше оригинала
    vpStage1.Height = (float)s1Height; // Высота в 4 раза меньше оригинала
    vpStage1.MinDepth = 0.0f;
    vpStage1.MaxDepth = 1.0f;

    // === ФИЗИЧЕСКОЕ СОЗДАНИЕ ХОЛСТА ДЛЯ СТАДИИ 2 (ИСПРАВЛЕНИЕ: ДЕЛАЕМ ПОЛНЫЙ РАЗМЕР) ===
    UINT s2Width = fileWidth;   // СТАЛО: ПОЛНЫЙ РАЗМЕР (Вместо fileWidth / 8)
    UINT s2Height = fileHeight; // СТАЛО: ПОЛНЫЙ РАЗМЕР (Вместо fileHeight / 8)

    if (g_pTexStretched)
    {
        D3D11_TEXTURE2D_DESC currentDesc2;
        g_pTexStretched->GetDesc(&currentDesc2);
        if (currentDesc2.Width != s2Width || currentDesc2.Height != s2Height)
        {
            if (g_pStageDown4SRV) { g_pStageDown4SRV->Release(); g_pStageDown4SRV = nullptr; }
            if (g_pStageDown4RTV) { g_pStageDown4RTV->Release(); g_pStageDown4RTV = nullptr; }
            g_pTexStretched->Release();
            g_pTexStretched = nullptr;
        }
    }

    if (g_pTexStretched == nullptr)
    {
        D3D11_TEXTURE2D_DESC descS2 = {};
        descS2.Width = s2Width;   // Теперь сюда придет полный размер 4128
        descS2.Height = s2Height; // Теперь сюда придет полный размер 3096
        descS2.MipLevels = 1;
        descS2.ArraySize = 1;
        descS2.Format = DXGI_FORMAT_R8G8B8A8_UNORM; // СТАЛО: Стандартный формат для BitBlt
```

```
    descS2.SampleDesc.Count = 1;
    descS2.SampleDesc.Quality = 0;
    descS2.Usage = D3D11_USAGE_DEFAULT;
    descS2.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;
    descS2.CPUAccessFlags = 0;
    descS2.MiscFlags = 0;

    HRESULT hr_s2 = g_pd3dDevice->CreateTexture2D(&descS2, nullptr, &g_pTexStretched);
    if (SUCCEEDED(hr_s2))
    {
        g_pd3dDevice->CreateRenderTargetView(g_pTexStretched, nullptr, &g_pRTVStretched);
        g_pd3dDevice->CreateShaderResourceView(g_pTexStretched, nullptr, &g_pSRVStretched);
    }
}

// === НАСТРОЙКА ОБЛАСТИ ПРОСМОТРА ДЛЯ СТАДИИ 2 (СЖАТИЕ 8x8) ===
D3D11_VIEWPORT vpStage2 = {};
vpStage2.TopLeftX = 0.0f;
vpStage2.TopLeftY = 0.0f;
vpStage2.Width = (float)s2Width;
vpStage2.Height = (float)s2Height;
vpStage2.MinDepth = 0.0f;
vpStage2.MaxDepth = 1.0f;

// ========================================================================
// ЭТАП I: ОДНОПРОХОДНЫЙ РАСЧЕТ И СБОРКА ШТОРКИ В ПОЛНЫЙ РАЗМЕР ФАЙЛА
// ========================================================================

// Загружаем в видеокарту чистую эталонную геометрию квада для скрытых пассов
SimpleVertex masterVerticesQuad[] =
{
    { { -1.0f,  1.0f, 0.0f },{ 0.0f, 0.0f } },
```

```
    { { 1.0f,  1.0f, 0.0f },{ 1.0f, 0.0f } },
    { { -1.0f, -1.0f, 0.0f },{ 0.0f, 1.0f } },
    { { 1.0f, -1.0f, 0.0f },{ 1.0f, 1.0f } },
  };
  g_pImmediateContext->UpdateSubresource(g_pVertexBuffer, 0, NULL, masterVerticesQuad, 0, 0);

  // Пасс А: Запускаем оригинальный монолитный дефектоскоп 2011 года в буфер Stage3
  D3D11_VIEWPORT vpS3 = { 0.0f, 0.0f, renderW, renderH, 0.0f, 1.0f };
  g_pImmediateContext->RSSetViewports(1, &vpS3);

  // Заполняем структуру констант для шейдера
  ShaderConstants cbDataLocal = {};
  cbDataLocal.width = renderW;
  cbDataLocal.height = renderH;
  cbDataLocal.d_width = 1.0f / renderW;
  cbDataLocal.d_height = 1.0f / renderH;
  cbDataLocal.splitX = 0.5f; // Шторка строго по центру full-size кадра
  g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbDataLocal, 0, 0);

  // ========================================================================
  // ПАСС НУЛЕВОЙ: ЗАПУСК ПЕРВОГО ШАГА СЖАТИЯ (БЛОКИ 4x4 В КАНАЛЫ R, G, B, A)
  // ========================================================================

  // === 1. КОНТРОЛЬ GPU: ОДНОКРАТНОЕ СКАЧИВАНИЕ ИСХОДНОГО ЦВЕТНОГО ОРИГИНАЛА ===
  if (g_bNeedCopyOriginal && g_pTexOriginal && g_pStagingOriginal && pRawPixels != NULL)
  {
    g_pImmediateContext->CopyResource(g_pStagingOriginal, g_pTexOriginal);
    D3D11_MAPPED_SUBRESOURCE mappedResource;
    if (SUCCEEDED(g_pImmediateContext->Map(g_pStagingOriginal, 0, D3D11_MAP_READ, 0,
&mappedResource)))
    {
      BYTE* pSrc = (BYTE*)mappedResource.pData;
      BYTE* pDst = (BYTE*)pRawPixels; // Цветной оригинал пишем в универсальный pRawPixels
```

```
        UINT rowSize = g_currentImgWidth * sizeof(UINT);
        for (UINT y = 0; y < g_currentImgHeight; y++)
        {
            memcpy(pDst + (y * rowSize), pSrc + (y * mappedResource.RowPitch), rowSize);
        }
        g_pImmediateContext->Unmap(g_pStagingOriginal, 0);
    }
    g_bNeedCopyOriginal = false;
  }

  // === 2. КОНТРОЛЬ GPU: ОДНОКРАТНОЕ СКАЧИВАНИЕ МОНОХРОМА (РЕЖИМ 2) ===
  if (g_bNeedCopyMonochrome && g_pTexMonochrome && g_pStagingMonochrome && pMonochromeBuffer != NULL)
  {
    g_pImmediateContext->CopyResource(g_pStagingMonochrome, g_pTexMonochrome);
    D3D11_MAPPED_SUBRESOURCE mappedResource;
    if (SUCCEEDED(g_pImmediateContext->Map(g_pStagingMonochrome, 0, D3D11_MAP_READ, 0, &mappedResource)))
    {
        BYTE* pSrc = (BYTE*)mappedResource.pData;
        BYTE* pDst = (BYTE*)pMonochromeBuffer; // Честный ЧБ слепок пишем строго в pMonochromeBuffer
        UINT rowSize = g_currentImgWidth * sizeof(UINT);
        for (UINT y = 0; y < g_currentImgHeight; y++)
        {
            memcpy(pDst + (y * rowSize), pSrc + (y * mappedResource.RowPitch), rowSize);
        }
        g_pImmediateContext->Unmap(g_pStagingMonochrome, 0);
    }
    g_bNeedCopyMonochrome = false; // Опускаем наш выделенный флаг
  }

  // === АППАРАТНЫЙ ПАСС GPU: ПЕРЕВОД ОРИГИНАЛА 1:1 В МОНОХРОМ ===
```

```
// Выполняем строго здесь, так как исходная текстура гарантированно готова к работе
  if (g_pRTVMonochrome && g_pTextureSRV)
  {
    float ClearColorMono[4] = { 0.0f, 0.0f, 0.0f, 1.0f };
    g_pImmediateContext->ClearRenderTargetView(g_pRTVMonochrome, ClearColorMono);
    g_pImmediateContext->OMSetRenderTargets(1, &g_pRTVMonochrome, nullptr);

    // Сетка под полный размер оригинала 1:1
    D3D11_VIEWPORT vpMono = { 0.0f, 0.0f, renderW, renderH, 0.0f, 1.0f };
    g_pImmediateContext->RSSetViewports(1, &vpMono);

    // Запускаем наш аппаратный ЧБ-шейдер по исходной текстуре
    g_pImmediateContext->PSSetShader(g_pPixelShaderMonochrome, nullptr, 0);
    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
    g_pImmediateContext->Draw(4, 0);

    // Сбрасываем цель рендера, чтобы освободить конвейер для пасса сжатия
    g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);
  }

  float ClearColorS1[4] = { 0.0f, 0.0f, 0.0f, 1.0f };
  g_pImmediateContext->ClearRenderTargetView(g_pRTVCompressed, ClearColorS1);
  g_pImmediateContext->OMSetRenderTargets(1, &g_pRTVCompressed, nullptr);
  g_pImmediateContext->RSSetViewports(1, &vpStage1);
  g_pImmediateContext->PSSetShader(g_pPixelShaderDown, nullptr, 0);
  g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
  g_pImmediateContext->Draw(4, 0);

  // Жестко отвязываем холст Первой Стадии от записи, закрывая его
  g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);

  // Разрываем связи после сжатия, подготавливая видеокарту к Пассу А
```

```
ID3D11ShaderResourceView* nullSRV_S1[] = { nullptr };
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV_S1);
g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);

// ========================================================================
// ПАСС ПЕРВЫЙ: ЗАПУСК ВТОРОГО ШАГА СЖАТИЯ (МАКРОСТРУКТУРА 8x8 НА БАЗЕ 4x4)
// ========================================================================
float ClearColorS2[4] = { 0.0f, 0.0f, 0.0f, 1.0f };
g_pImmediateContext->ClearRenderTargetView(g_pStageDown4RTV, ClearColorS2);
g_pImmediateContext->OMSetRenderTargets(1, &g_pStageDown4RTV, nullptr);
g_pImmediateContext->RSSetViewports(1, &vpStage2);
g_pImmediateContext->PSSetShader(g_pPixelShaderDown4x4, nullptr, 0);

// Подаем на вход текстуру с результатами первого шага (4x4)
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pStageDown4SRV);
g_pImmediateContext->Draw(4, 0);

// Разрываем связи после второго шага сжатия
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV_S1);
g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);

// ========================================================================
// ПАСС ВТОРОЙ: АППАРАТНОЕ РАСТЯЖЕНИЕ МАКРОСТРУКТУРЫ (64 РАЗНЫХ ПИКСЕЛЯ!)
// ========================================================================
float ClearColorStage2[4] = { 0.0f, 0.0f, 0.0f, 1.0f };
g_pImmediateContext->ClearRenderTargetView(g_pRTVStretched, ClearColorStage2);
g_pImmediateContext->OMSetRenderTargets(1, &g_pRTVStretched, nullptr);
g_pImmediateContext->RSSetViewports(1, &vpStage2);

// === НАЧАЛО ИЗМЕНЕНИЯ ===
 // Отключаем тест глубины и трафарета, чтобы пиксели заглушек гарантированно записались
g_pImmediateContext->OMSetDepthStencilState(nullptr, 0);
```

```
    // Отключаем смешивание цветов (Blend State), переводим в режим обычной перезаписи
    g_pImmediateContext->OMSetBlendState(nullptr, nullptr, 0xffffffff);

    // Включаем наш автономный генератор вершин для теста заглушек
    g_pImmediateContext->VSSetShader(g_pDiagnosticVS, nullptr, 0);

    g_pImmediateContext->PSSetShader(g_pPixelShaderStage2, nullptr, 0);
    g_pImmediateContext->PSSetShaderResources(1, 1, &g_pSRVCompressed);
    g_pImmediateContext->Draw(4, 0);

    // ВАЖНО: Возвращаем базовый вершинный шейдер плеера на место, чтобы Пасс Ноль не ломался!
    g_pImmediateContext->VSSetShader(g_pVertexShader, nullptr, 0);

    // Жестко отвязываем холст записи, закрывая его перед скачиванием в CPU
    g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);

    // Чисто разрываем связи после растяжения
    g_pImmediateContext->PSSetShaderResources(1, 1, nullSRV);
    g_pImmediateContext->OMSetRenderTargets(1, nullRTV, nullptr);

    g_pImmediateContext->RSSetViewports(1, &vpS3);

    g_pImmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearColorBlack);
    g_pImmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);
    g_pImmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);
    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);     // t0: Исходный оригинал кадра 1:1
    g_pImmediateContext->PSSetShaderResources(4, 1, &g_pSRVStretched);      // t4: Растянутое плавное поле из Stage2

    g_pImmediateContext->Draw(4, 0);
```

```
  // Чисто разрываем связи Пасса А
  // ========================================================================
  // НОВЫЙ БЛОК: СКАЧИВАНИЕ ГОТОВОГО СЛЕПКА GPU ДЛЯ КОНТРОЛЯ В РЕЖИМЕ 4
  // ========================================================================
  if (g_bNeedCopyStretched && g_pSRVStretched && g_pStagingStretched)
  {
    ID3D11Resource* pResSource = nullptr;
    g_pSRVStretched->GetResource(&pResSource);

    if (pResSource)
    {
      g_pImmediateContext->CopyResource(g_pStagingStretched, pResSource);
      pResSource->Release();
    }
  }
  g_bNeedCopyStretched = false;

  g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
  g_pImmediateContext->OMSetRenderTargets(1, nullRTV, NULL);

  // Пасс Б: Сборка финальной шторки "До / После" в Полноразмерный Мастер-Буфер
  g_pImmediateContext->ClearRenderTargetView(g_pStageMasterRTV, ClearColorBlack);
  g_pImmediateContext->OMSetRenderTargets(1, &g_pStageMasterRTV, NULL);
  g_pImmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);
  g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV); // t0: Оригинал кадра

  if (g_bUseCPUProcessing && g_pCpuTextureSRV) {
    g_pImmediateContext->PSSetShaderResources(1, 1, &g_pCpuTextureSRV); // t1: Красивый CPU-результат
  }
  else {
    g_pImmediateContext->PSSetShaderResources(1, 1, &g_pStage3SRV);  // t1: Результат дефектоскопа от GPU
```

```
}
g_pImmediateContext->Draw(4, 0);

// Полный сброс скрытого этапа вычислений
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pImmediateContext->PSSetShaderResources(1, 1, nullSRV);
g_pImmediateContext->OMSetRenderTargets(1, nullRTV, NULL);

// ЖЕСТКАЯ ОЧИСТКА ШЕЙДЕРА МЕЖДУ ПАССАМИ (Убирает фантомное двоение масштабов)
g_pImmediateContext->PSSetShader(NULL, NULL, 0);

// ========================================================================
// ЭТАП II: ВЫВОД ГОТОВОГО РЕЗУЛЬТАТА НА ЭКРАН С УЧЕТОМ МАСШТАБА И СКРОЛЛИНГА
// ========================================================================

g_pImmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearColorGrey);
g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

D3D11_VIEWPORT vpFull = { 0.0f, 0.0f, (float)g_wndW, (float)g_wndH, 0.0f, 1.0f };
g_pImmediateContext->RSSetViewports(1, &vpFull);

float xLeft = -1.0f, xRight = 1.0f;
float yTop = 1.0f, yBottom = -1.0f;

if (g_bFitToWindow)
{
   float windowAspect = (float)g_wndW / (float)g_wndH;
   float imageAspect = (float)fileWidth / (float)fileHeight;

   if (imageAspect > windowAspect) {
      float scaleY = windowAspect / imageAspect;
      yTop = scaleY; yBottom = -scaleY;
```

```
    }
    else {
        float scaleX = imageAspect / windowAspect;
        xLeft = -scaleX; xRight = scaleX;
    }
}
else
{
    float targetW = (float)fileWidth / g_zoomDivider;
    float targetH = (float)fileHeight / g_zoomDivider;

    float maxScrollX = (targetW > (float)g_wndW) ? (targetW - (float)g_wndW) : 0.0f;
    float maxScrollY = (targetH > (float)g_wndH) ? (targetH - (float)g_wndH) : 0.0f;

    float currentOffsetX = g_scrollRatioX * maxScrollX;
    float currentOffsetY = g_scrollRatioY * maxScrollY;

    float posX = -currentOffsetX;
    float posY = -currentOffsetY;

    xLeft = (posX / (float)g_wndW) * 2.0f - 1.0f;
    xRight = ((posX + targetW) / (float)g_wndW) * 2.0f - 1.0f;
    yTop = 1.0f - (posY / (float)g_wndH) * 2.0f;
    yBottom = 1.0f - ((posY + targetH) / (float)g_wndH) * 2.0f;
}

g_currentVertices[0] = { { xLeft,  yTop,    0.0f },{ 0.0f, 0.0f } };
g_currentVertices[1] = { { xRight, yTop,    0.0f },{ 1.0f, 0.0f } };
g_currentVertices[2] = { { xLeft,  yBottom, 0.0f },{ 0.0f, 1.0f } };
g_currentVertices[3] = { { xRight, yBottom, 0.0f },{ 1.0f, 1.0f } };

g_pImmediateContext->UpdateSubresource(g_pVertexBuffer, 0, NULL, g_currentVertices, 0, 0);
```

```
    g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
    g_pImmediateContext->PSSetShader(g_pPixelShaderCopy, NULL, 0);

    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pStageMasterSRV);
    g_pImmediateContext->Draw(4, 0);

    g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
    g_pSwapChain->Present(0, 0);
}

// === НОВЫЙ БЕЗОПАСНЫЙ КОД: ФУНКЦИЯ ДВУХЭКРАННОГО ВЫВОДА "ДО / ПОСЛЕ" ===
void RenderSplit()
{
    if (g_pRenderTargetView == NULL) return;

    // 1. Общие настройки сетки экрана
    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;
    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

    // 2. Получаем размеры оригинального файла для шейдеров
    ShaderConstants cbData;
    cbData.width = 2.0f;
    cbData.height = 2.0f;
    if (g_pTextureSRV)
    {
        ID3D11Resource* pRes = NULL;
        g_pTextureSRV->GetResource(&pRes);
        if (pRes)
```

```
    {
        ID3D11Texture2D* pT2D = (ID3D11Texture2D*)pRes;
        D3D11_TEXTURE2D_DESC d;
        pT2D->GetDesc(&d);
        cbData.width = (float)d.Width;
        cbData.height = (float)d.Height;
        pRes->Release();
    }
}
cbData.d_width = 1.0f / cbData.width;
cbData.d_height = 1.0f / cbData.height;
g_pImmediateContext->UpdateSubresource(g_pConstantBuffer, 0, NULL, &cbData, 0, 0);

g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
g_pImmediateContext->PSSetConstantBuffers(0, 1, &g_pConstantBuffer);
g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState);

ID3D11ShaderResourceView* nullSRV[] = { NULL };

// --- ВЫПОЛНЯЕМ СКРЫТЫЕ ПРОХОДЫ АНАЛИЗА (1, 2 и 3) ---
float ClearBlack[] = { 0.0f, 0.0f, 0.0f, 1.0f };

// Проход 1 (Stage1)
g_pImmediateContext->ClearRenderTargetView(g_pRTVCompressed, ClearBlack);
g_pImmediateContext->OMSetRenderTargets(1, &g_pRTVCompressed, NULL);
g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
g_pImmediateContext->Draw(4, 0);

// Проход 2 (Stage2)
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pImmediateContext->ClearRenderTargetView(g_pRTVStretched, ClearBlack);
```

```
g_pImmediateContext->OMSetRenderTargets(1, &g_pRTVStretched, NULL);
g_pImmediateContext->PSSetShader(g_pPixelShaderStage2, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pSRVCompressed);
g_pImmediateContext->Draw(4, 0);

// Проход 3 (Stage3)
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
g_pImmediateContext->ClearRenderTargetView(g_pStage3RTV, ClearBlack);
g_pImmediateContext->OMSetRenderTargets(1, &g_pStage3RTV, NULL);
g_pImmediateContext->PSSetShader(g_pPixelShaderStage3, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pSRVStretched);
g_pImmediateContext->Draw(4, 0);

// --- ВЫВОД НА ЭКРАН В ДВА ПОРТА ПРОСМОТРА ---
g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
float ClearGrey[] = { 0.75f, 0.75f, 0.75f, 1.0f };
g_pImmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearGrey);
g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

// Жесткие безопасные размеры для деления окна пополам
float sideWidth = 400.0f;
float sideHeight = 600.0f;

// ЛЕВАЯ ПОЛОВИНА: Сырой оригинал
D3D11_VIEWPORT vpL = {};
vpL.Width = sideWidth;
vpL.Height = sideHeight;
vpL.MaxDepth = 1.0f;
g_pImmediateContext->RSSetViewports(1, &vpL);

g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);
g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
```

```
    g_pImmediateContext->Draw(4, 0);
    g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);

    // ПРАВАЯ ПОЛОВИНА: Финальный сглаженный результат из Stage3
    D3D11_VIEWPORT vpR = {};
    vpR.Width = sideWidth;
    vpR.Height = sideHeight;
    vpR.TopLeftX = sideWidth; // Сдвиг вправо
    vpR.MaxDepth = 1.0f;
    g_pImmediateContext->RSSetViewports(1, &vpR);

    g_pImmediateContext->PSSetShader(g_pPixelShaderFinal, NULL, 0);
    // >>> ВНЕДРЯЕМ АДАПТИВНЫЙ ВЫБОР: GPU ИЛИ КРАСИВАЯ КАРТИНКА CPU
    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV); // Слева Оригинал

    // Гибкое переключение источника для правой половины шторки
    if (g_bUseCPUProcessing && g_pCpuTextureSRV)
    {
        // Включен Программный режим — подаем текстуру, рассчитанную процессором
        g_pImmediateContext->PSSetShaderResources(1, 1, &g_pCpuTextureSRV);
    }
    else
    {
        // Включен аппаратный режим — подаем результат Stage3 от видеокарты
        g_pImmediateContext->PSSetShaderResources(1, 1, &g_pStage3SRV);
    }

    // Сброс и вывод кадра
    g_pImmediateContext->PSSetShaderResources(0, 1, nullSRV);
    g_pSwapChain->Present(0, 0);
}
```

```
// Безопасное освобождение памяти при выходе
void CleanupDevice()
{
    if (g_pImmediateContext) g_pImmediateContext->ClearState();

    if (g_pSRVCompressed) { g_pSRVCompressed->Release(); g_pSRVCompressed = NULL; }
    if (g_pRTVCompressed) { g_pRTVCompressed->Release(); g_pRTVCompressed = NULL; }
    if (g_pSRVStretched) { g_pSRVStretched->Release(); g_pSRVStretched = NULL; } // НОВОЕ: Очистка ресурса чтения Stage2
    if (g_pRTVStretched) { g_pRTVStretched->Release(); g_pRTVStretched = NULL; } // НОВОЕ: Очистка холста записи Stage2
    if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }
    if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }
    if (g_pStageMasterSRV) { g_pStageMasterSRV->Release(); g_pStageMasterSRV = NULL; }
    if (g_pStageMasterRTV) { g_pStageMasterRTV->Release(); g_pStageMasterRTV = NULL; }
    // Чистое освобождение ресурсов программного режима CPU
    if (g_pCpuTextureSRV) { g_pCpuTextureSRV->Release(); g_pCpuTextureSRV = NULL; }
    if (g_pCpuTexture) { g_pCpuTexture->Release();    g_pCpuTexture = NULL; }
    if (g_pSamplerState) { g_pSamplerState->Release(); g_pSamplerState = NULL; }
    if (g_pConstantBuffer) { g_pConstantBuffer->Release(); g_pConstantBuffer = NULL; }
    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }
    if (g_pVertexBuffer) { g_pVertexBuffer->Release(); g_pVertexBuffer = NULL; }
    if (g_pVertexLayout) { g_pVertexLayout->Release(); g_pVertexLayout = NULL; }
    if (g_pPixelShader) { g_pPixelShader->Release(); g_pPixelShader = NULL; }
    if (g_pPixelShaderStage2) { g_pPixelShaderStage2->Release(); g_pPixelShaderStage2 = NULL; } // НОВОЕ: Очистка 4-го шейдера анализа
    if (g_pPixelShaderStage3) { g_pPixelShaderStage3->Release(); g_pPixelShaderStage3 = NULL; } // НОВОЕ: Очистка 5-го шейдера фильтрации
    if (g_pPixelShaderFinal) { g_pPixelShaderFinal->Release(); g_pPixelShaderFinal = NULL; } // НОВОЕ: Очистка 6-го шейдера цветокоррекции
    if (g_pPixelShaderCopy) { g_pPixelShaderCopy->Release(); g_pPixelShaderCopy = NULL; }
    if (g_pPixelShaderMonochrome) { g_pPixelShaderMonochrome->Release(); g_pPixelShaderMonochrome = NULL; }
    if (g_pVertexShader) { g_pVertexShader->Release(); g_pVertexShader = NULL; }
```

```
    if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }
    if (g_pSwapChain) { g_pSwapChain->Release(); g_pSwapChain = NULL; }
    if (g_pImmediateContext) { g_pImmediateContext->Release(); g_pImmediateContext = NULL; }
    if (g_pd3dDevice) { g_pd3dDevice->Release(); g_pd3dDevice = NULL; }
}

// === НОВЫЙ КОД: ФУНКЦИЯ СОЗДАНИЯ СИСТЕМНОГО МЕНЮ ===
void CreateAppMenu(HWND hwnd)
{
    HMENU hMenu = CreateMenu();        // Главная горизонтальная полоса
    HMENU hFileMenu = CreatePopupMenu(); // Выпадающее меню "Файл"
    HMENU hViewMenu = CreatePopupMenu(); // Выпадающее меню "Вид"
    HMENU hHelpMenu = CreatePopupMenu(); // Выпадающее меню "Справка"

    // Заполняем меню "Файл"
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_VIDEO, L"Открыть видео поток (AVI/MP4)...");
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_OPEN_IMAGE, L"Открыть кадр/картинку (BMP/PNG)...");
    // === НОВОЕ: ДОБАВЛЯЕМ УПРАВЛЕНИЕ РЕЖИМАМИ СРАВНЕНИЯ ===
    AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_CPU, L"Программный режим (CPU)");
    // === ВНЕДРЕНИЕ: КАСКАДНОЕ ПОДМЕНЮ ДИСКРЕТНЫХ МАСШТАБОВ ===
    HMENU hZoomSubMenu = CreateMenu();
    // Формируем 6 эталонных вариантов масштабирования
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_FIT, L"Вписать в кадр");
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_1, L"Масштаб 1:1 (Оригинал)");
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_2, L"Масштаб 1:2");
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_4, L"Масштаб 1:4");
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_8, L"Масштаб 1:8");
    AppendMenuW(hZoomSubMenu, MF_STRING, IDM_ZOOM_16, L"Масштаб 1:16");
```

```
    // Встраиваем всплывающее подменю Масштаба внутрь нашего родительского меню "Вид" [INDEX_146]
    AppendMenuW(hViewMenu, MF_POPUP, (UINT_PTR)hZoomSubMenu, L"Масштаб");

    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
    AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL); // Разделительная линия
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_SAVE_RESULT, L"Записать преобразованный файл...");
    AppendMenuW(hFileMenu, MF_SEPARATOR, 0, NULL);
    AppendMenuW(hFileMenu, MF_STRING, IDM_FILE_EXIT, L"Выход");

    // Заполняем меню "Вид"
    AppendMenuW(hViewMenu, MF_STRING | MF_CHECKED, IDM_VIEW_RESULT_ONLY, L"Только обработанный результат");
    AppendMenuW(hViewMenu, MF_STRING, IDM_VIEW_SPLIT, L"Режим \"До / После\" (Разделение экрана)");

    // Заполняем меню "Справка"
    AppendMenuW(hHelpMenu, MF_STRING, IDM_HELP_ABOUT, L"О программе...");

    // Прикрепляем выпадающие списки к главной полосе меню
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, L"Файл");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hViewMenu, L"Вид");
    AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hHelpMenu, L"Справка");

    // Физически вешаем меню на наше окно программы
    SetMenu(hwnd, hMenu);
}
// === КОНЕЦ НОВОГО КОДА ===

#include <wincodec.h> // Подключаем заголовки системного декодера WIC

// === БЕЗОПАСНАЯ ЗАГРУЗКА ЛЮБЫХ КАРТИНОК ЧЕРЕЗ WIC НА ВИДЕОКАРТУ ===
HRESULT LoadTextureFromFile(const WCHAR* szFileName)
```

```
{
    HRESULT hr = S_OK;

    // Переключаем флаги: принудительно просим видеокарту сделать
    // один свежий слепок нового файла для панели диагностики
    g_bNeedCopyOriginal = true;
    g_bNeedCopyMonochrome = true;
    g_bNeedCopyStage3 = true;
    g_bNeedCopyCompressed = true;

    // 1. Создаем фабрику декодеров WIC
    IWICImagingFactory* pWICFactory = NULL;
    hr = CoCreateInstance(CLSID_WICImagingFactory, NULL, CLSCTX_INPROC_SERVER,
__uuidof(IWICImagingFactory), (LPVOID*)&pWICFactory);
    if (FAILED(hr)) return hr;

    // 2. Открываем файл картинки на диске
    IWICBitmapDecoder* pDecoder = NULL;
    hr = pWICFactory->CreateDecoderFromFilename(szFileName, NULL, GENERIC_READ,
WICDecodeMetadataCacheOnDemand, &pDecoder);
    if (FAILED(hr)) { pWICFactory->Release(); return hr; }

    // 3. Берем самый первый кадр из файла
    IWICBitmapFrameDecode* pFrame = NULL;
    hr = pDecoder->GetFrame(0, &pFrame);
    if (FAILED(hr)) { pDecoder->Release(); pWICFactory->Release(); return hr; }

    // 4. Принудительно конвертируем пиксели в формат RGBA 32-бит
    IWICFormatConverter* pConverter = NULL;
    hr = pWICFactory->CreateFormatConverter(&pConverter);
    if (FAILED(hr)) { pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }
```

```
   hr = pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppPBGRA, WICBitmapDitherTypeNone, NULL, 0.0f, WICBitmapPaletteTypeCustom);
   if (FAILED(hr)) { pConverter->Release(); pFrame->Release(); pDecoder->Release(); pWICFactory->Release(); return hr; }

   // 5. Узнаем физические размеры загруженной картинки
   UINT imgWidth = 0, imgHeight = 0;
   pConverter->GetSize(&imgWidth, &imgHeight);

   // === НАЧАЛО ИЗМЕНЕНИЯ ===
   // Вычисляем новые размеры, кратные 8
   UINT paddedWidth = ((imgWidth + 7) / 8) * 8;
   UINT paddedHeight = ((imgHeight + 7) / 8) * 8;

   // Выделяем буфер под расширенный кадр и временный под сырые пиксели
   pPixelsBuffer = new UINT[paddedWidth * paddedHeight];
   pRawPixels = new UINT[imgWidth * imgHeight];

   hr = pConverter->CopyPixels(NULL, imgWidth * sizeof(UINT), imgWidth * imgHeight * sizeof(UINT), (BYTE*)pRawPixels);

   if (SUCCEEDED(hr))
   {
      // Заполняем расширенный буфер с дублированием краев
      for (UINT y = 0; y < paddedHeight; y++)
      {
         UINT srcY = (y < imgHeight) ? y : (imgHeight - 1);
         for (UINT x = 0; x < paddedWidth; x++)
         {
            UINT srcX = (x < imgWidth) ? x : (imgWidth - 1);
            pPixelsBuffer[y * paddedWidth + x] = pRawPixels[srcY * imgWidth + srcX];
            if (::pRawPixels != nullptr && srcY < imgHeight && srcX < imgWidth)
            {
```

```
        ::pRawPixels[srcY * imgWidth + srcX] = pRawPixels[srcY * imgWidth + srcX];
      }
    }
}

// 6. Если старая основная текстура уже была в памяти — чисто освобождаем её
if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }

// 7. Создаем новую текстуру прямо в видеопамяти под размеры нашей картинки
D3D11_TEXTURE2D_DESC desc = {};
desc.Width = imgWidth;
desc.Height = imgHeight;
desc.MipLevels = 1;
desc.ArraySize = 1;
desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
desc.SampleDesc.Count = 1;
desc.Usage = D3D11_USAGE_DEFAULT;
desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

D3D11_SUBRESOURCE_DATA initData = {};
initData.pSysMem = pPixelsBuffer;
initData.SysMemPitch = paddedWidth * sizeof(UINT);

ID3D11Texture2D* pTexture = NULL;
desc.Width = paddedWidth;
desc.Height = paddedHeight;
hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);
if (SUCCEEDED(hr))
{
   hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);
   pTexture->Release();
```

```
        g_currentImgWidth = paddedWidth;   // Строго paddedWidth (184)! Никаких imgWidth.
        g_currentImgHeight = paddedHeight; // Строго paddedHeight (184)!
        // Выделяем память для Режима 4 на основе точных размеров, принятых плеромФ
        if (g_currentImgWidth > 0 && g_currentImgHeight > 0)
        {
           if (pMonochromeBuffer != nullptr) { delete[] pMonochromeBuffer; pMonochromeBuffer =
nullptr; }
           pMonochromeBuffer = new UINT[g_currentImgWidth * g_currentImgHeight];

           if (::pStretchedBuffer != nullptr) { delete[] ::pStretchedBuffer; ::pStretchedBuffer = nullptr; }
           ::pStretchedBuffer = new UINT[g_currentImgWidth * g_currentImgHeight];
        }
        // === НОВОЕ: Выделяем память под эталонные буферы алгоритмов CPU ===
        if (cpu_pRawPixels != nullptr) { delete[] cpu_pRawPixels; cpu_pRawPixels = nullptr; }
        cpu_pRawPixels = new UINT[g_currentImgWidth * g_currentImgHeight];

        if (cpu_pMonochromeBuffer != nullptr) { delete[] cpu_pMonochromeBuffer;
cpu_pMonochromeBuffer = nullptr; }
        cpu_pMonochromeBuffer = new UINT[g_currentImgWidth * g_currentImgHeight];

        if (cpu_pCompressed4x != nullptr) { delete[] cpu_pCompressed4x; cpu_pCompressed4x = nullptr;
}
        cpu_pCompressed4x = new UINT[g_currentImgWidth * g_currentImgHeight];

        if (cpu_pDoubleCompressed16x != nullptr) { delete[] cpu_pDoubleCompressed16x;
cpu_pDoubleCompressed16x = nullptr; }
        cpu_pDoubleCompressed16x = new UINT[g_currentImgWidth * g_currentImgHeight];

        if (cpu_pStretchedBuffer != nullptr) { delete[] cpu_pStretchedBuffer; cpu_pStretchedBuffer =
nullptr; }
        cpu_pStretchedBuffer = new UINT[g_currentImgWidth * g_currentImgHeight];

        if (cpu_pDifferenceBuffer != nullptr) { delete[] cpu_pDifferenceBuffer; cpu_pDifferenceBuffer =
nullptr; }
```

```
    cpu_pDifferenceBuffer = new UINT[g_currentImgWidth * g_currentImgHeight];
}

// === НОВЫЙ КОД: ЗАПУСК ПРОГРАММНОЙ ОБРАБОТКИ CPU ПРИ ОТКРЫТИИ ФАЙЛА ===
if (SUCCEEDED(hr))
{
    UINT* pCpuOutPixels = new UINT[paddedWidth * paddedHeight];

    // Запускаем расчет на процессоре по НАСТОЯЩЕМУ исходному размеру кадра
    if (SUCCEEDED(ApplyCpuFilter(pPixelsBuffer, paddedWidth, paddedHeight, pCpuOutPixels)))
    {
        // Описываем параметры для текстуры процессора (тоже строго 184x184)
        D3D11_TEXTURE2D_DESC cpuDesc = {};
        cpuDesc.Width = paddedWidth;
        cpuDesc.Height = paddedHeight;
        cpuDesc.MipLevels = 1;
        cpuDesc.ArraySize = 1;
        cpuDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
        cpuDesc.SampleDesc.Count = 1;
        cpuDesc.Usage = D3D11_USAGE_DEFAULT;
        cpuDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

        D3D11_SUBRESOURCE_DATA cpuInitData = {};
        cpuInitData.pSysMem = pCpuOutPixels;
        cpuInitData.SysMemPitch = paddedWidth * sizeof(UINT);

        // Физически создаем холст результатов CPU на видеокарте
        ID3D11Texture2D* pCpuTex = NULL;
        if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&cpuDesc, &cpuInitData, &pCpuTex)))
        {
            g_pd3dDevice->CreateShaderResourceView(pCpuTex, NULL, &g_pCpuTextureSRV);
            pCpuTex->Release();
```

```
        }
    }

    // Освобождаем временный массив результатов процессора из ОЗУ
    delete[] pCpuOutPixels;
}

// ========================================================================
// СИНХРОНИЗАЦИЯ С КОДОМ 2011 ГОДА: ДИНАМИЧЕСКИЙ РЕЗАЙЗ СКРЫТЫХ БУФЕРОВ
// ========================================================================
if (SUCCEEDED(hr))
{
    // Чисто освобождаем все старые скрытые буферы из видеопамяти [INDEX_45, INDEX_46]
    if (g_pRTVCompressed) { g_pRTVCompressed->Release(); g_pRTVCompressed = NULL; }
    if (g_pSRVCompressed) { g_pSRVCompressed->Release(); g_pSRVCompressed = NULL; }
    if (g_pRTVStretched) { g_pRTVStretched->Release(); g_pRTVStretched = NULL; }
    if (g_pSRVStretched) { g_pSRVStretched->Release(); g_pSRVStretched = NULL; }
    if (g_pStage3RTV) { g_pStage3RTV->Release(); g_pStage3RTV = NULL; }
    if (g_pStage3SRV) { g_pStage3SRV->Release(); g_pStage3SRV = NULL; }

    // Вычисляем правильные пирамидальные размеры сжатия [INDEX_79]
    UINT stage1Width = (imgWidth / 4 > 0) ? imgWidth / 4 : 1;
    UINT stage1Height = (imgHeight / 4 > 0) ? imgHeight / 4 : 1;

    UINT stage2Width = imgWidth;
    UINT stage2Height = imgHeight;

    // --- Создаем буфер Stage 1 (Меньше в 4 раза под блоки 4x4) ---
    D3D11_TEXTURE2D_DESC descS1 = {};
    descS1.Width = stage1Width;
    descS1.Height = stage1Height;
    descS1.MipLevels = 1;
```

```
descS1.ArraySize = 1;
descS1.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
descS1.SampleDesc.Count = 1;
descS1.Usage = D3D11_USAGE_DEFAULT;
descS1.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pTexS1 = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS1, NULL, &pTexS1))) {
   g_pd3dDevice->CreateRenderTargetView(pTexS1, NULL, &g_pRTVCompressed);
   g_pd3dDevice->CreateShaderResourceView(pTexS1, NULL, &g_pSRVCompressed);
   pTexS1->Release();
}

// --- Создаем буфер Stage 2 (ИСПРАВЛЕНИЕ: ДЕЛАЕМ ПОЛНОРАЗМЕРНЫМ ПОД СЕТКУ 1:1) --
-
D3D11_TEXTURE2D_DESC descS2 = {};
descS2.Width = imgWidth;   // СТАЛО: ПОЛНЫЙ РАЗМЕР (ВМЕСТО stage2Width)
descS2.Height = imgHeight; // СТАЛО: ПОЛНЫЙ РАЗМЕР (ВМЕСТО stage2Height)
descS2.MipLevels = 1;
descS2.ArraySize = 1;
descS2.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
descS2.SampleDesc.Count = 1;
descS2.SampleDesc.Quality = 0;
descS2.Usage = D3D11_USAGE_DEFAULT;
descS2.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

ID3D11Texture2D* pTexS2 = NULL;
if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS2, NULL, &pTexS2))) {
   g_pd3dDevice->CreateRenderTargetView(pTexS2, NULL, &g_pRTVStretched);
   g_pd3dDevice->CreateShaderResourceView(pTexS2, NULL, &g_pSRVStretched);
   pTexS2->Release();
}
```

```
        // --- Создаем буфер Stage 3 (ПОЛНОРАЗМЕРНЫЙ под гладкий финальный кадр) ---
        // --- ИСПРАВЛЕНИЕ: ПЕРЕВОДИМ STAGE 3 В НАСТОЯЩИЙ HDR-ФОРМАТ С ПЛАВАЮЩЕЙ ТОЧКОЙ ---
        D3D11_TEXTURE2D_DESC descS3 = {};
        descS3.Width = imgWidth;
        descS3.Height = imgHeight;
        descS3.MipLevels = 1;
        descS3.ArraySize = 1;
        descS3.Format = DXGI_FORMAT_R16G16B16A16_FLOAT; // ВКЛЮЧИЛИ 16-БИТНЫЙ FLOAT ДЛЯ МАТЕМАТИКИ ИЗ 2011 ГОДА!
        descS3.SampleDesc.Count = 1;
        descS3.Usage = D3D11_USAGE_DEFAULT;
        descS3.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

        ID3D11Texture2D* pTexS3 = NULL;
        if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descS3, NULL, &pTexS3))) {
            g_pd3dDevice->CreateRenderTargetView(pTexS3, NULL, &g_pStage3RTV);
            g_pd3dDevice->CreateShaderResourceView(pTexS3, NULL, &g_pStage3SRV);
            pTexS3->Release();
        }
    }
    if (g_pStageMasterRTV) { g_pStageMasterRTV->Release(); g_pStageMasterRTV = NULL; }
    if (g_pStageMasterSRV) { g_pStageMasterSRV->Release(); g_pStageMasterSRV = NULL; }

    D3D11_TEXTURE2D_DESC descMaster = {};
    descMaster.Width = imgWidth;
    descMaster.Height = imgHeight;
    descMaster.MipLevels = 1;
    descMaster.ArraySize = 1;
    descMaster.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    descMaster.SampleDesc.Count = 1;
    descMaster.Usage = D3D11_USAGE_DEFAULT;
```

```
    descMaster.BindFlags = D3D11_BIND_RENDER_TARGET | D3D11_BIND_SHADER_RESOURCE;

    ID3D11Texture2D* pTexMaster = NULL;
    if (SUCCEEDED(g_pd3dDevice->CreateTexture2D(&descMaster, NULL, &pTexMaster))) {
      g_pd3dDevice->CreateRenderTargetView(pTexMaster, NULL, &g_pStageMasterRTV);
      g_pd3dDevice->CreateShaderResourceView(pTexMaster, NULL, &g_pStageMasterSRV);
      pTexMaster->Release();
    }
  }
  // === ИСПРАВЛЕНИЕ: ЖЕСТКОЕ И БЕЗОПАСНОЕ ИЗМЕНЕНИЕ РАЗМЕРОВ ОКНА ПРИ ОТКРЫТИИ ФАЙЛА ===
  if (SUCCEEDED(hr))
  {
    HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);
    if (hMainWnd)
    {
      // Рассчитываем габариты окна с учетом меню и заголовка Windows
      RECT rc = { 0, 0, (LONG)imgWidth, (LONG)imgHeight };
      AdjustWindowRect(&rc, WS_OVERLAPPEDWINDOW, TRUE);

      // Физически раздвигаем окно под размер картинки!
      MoveWindow(hMainWnd, 100, 100, rc.right - rc.left, rc.bottom - rc.top, TRUE);
    }
  }

  // ========================================================================
  // НАСТРОЙКА ДИАПАЗОНОВ С КРОЛЛИНГА ПОД РАЗМЕР КАРТИНКИ
  // ========================================================================
  if (SUCCEEDED(hr))
  {
    // Сохраняем размеры файла для обработчика прокрутки
    g_currentImgWidth = imgWidth;
    g_currentImgHeight = imgHeight;
```

```
    g_scrollX = 0;
    g_scrollY = 0;

    // === АВТОМАТИЧЕСКИЙ РАСЧЕТ КОЭФФИЦИЕНТОВ СКОРОСТИ СКРОЛЛИНГА ===
    g_scrollMultiplierX = (imgWidth > (UINT)g_wndW) ? ((float)imgWidth / g_wndW) : 1.0f;
    g_scrollMultiplierY = (imgHeight > (UINT)g_wndH) ? ((float)imgHeight / g_wndH) : 1.0f;

    HWND hMainWnd = FindWindowW(L"MyShaderVideoPlayerClass", NULL);
    if (hMainWnd)
    {
        // Настраиваем горизонтальный ползунок (X)
        SCROLLINFO siX = {};
        siX.cbSize = sizeof(SCROLLINFO);
        siX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
        siX.nMin = 0;
        siX.nMax = imgWidth;
        siX.nPage = 800; // Шаг страницы равен размеру окна
        siX.nPos = 0;
        SetScrollInfo(hMainWnd, SB_HORZ, &siX, TRUE);

        // Настраиваем вертикальный ползунок (Y)
        SCROLLINFO siY = {};
        siY.cbSize = sizeof(SCROLLINFO);
        siY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
        siY.nMin = 0;
        siY.nMax = imgHeight;
        siY.nPage = 600;
        siY.nPos = 0;
        SetScrollInfo(hMainWnd, SB_VERT, &siY, TRUE);
    }
}
```

```
g_srcImgWidth = imgWidth;
g_srcImgHeight = imgHeight;

// Вычисляем новые размеры, кратные 4
g_compressedWidth = ((imgWidth + 3) / 4);
g_compressedHeight = ((imgHeight + 3) / 4);

g_paddedImgWidth = g_compressedWidth * 4;
g_paddedImgHeight = g_compressedHeight * 4;

// === ЗАПОЛНЕНИЕ ОТЛАДОЧНОГО МАССИВА ПРИ ОТКРЫТИИ ФАЙЛА ===
if (g_currentImgWidth > 0 && g_currentImgHeight > 0)
{
   // 1. Освобождаем старую память массива, если она была выделена ранее
   if (pRawPixels != nullptr) { delete[] pRawPixels; pRawPixels = nullptr; }

   // 2. Выделяем чистую оперативную память строго под габариты новой картинки
   pRawPixels = new UINT[g_currentImgWidth * g_currentImgHeight];

   // Дальше идет ваш родной цикл без изменений
   for (UINT y = 0; y < g_currentImgHeight; y++)
   {
      for (UINT x = 0; x < g_currentImgWidth; x++)
      {
         UINT idx = y * g_currentImgWidth + x;
         if (pPixelsBuffer)
         {
            pRawPixels[idx] = pPixelsBuffer[idx];
         }
      }
   }
}
```

```
// === ЭТАЛОННЫЙ РАСЧЕТ МАКРОСТРУКТУРЫ 8x8 (ИЗ 2011 ГОДА) ДЛЯ РЕЖИМА 3 ===
if (pRawPixels && imgWidth > 0 && imgHeight > 0)
{
    UINT compW = imgWidth / 8;
    UINT compH = imgHeight / 8;

    // 1. Безопасно перевыделяем память под сжатый буфер в ОЗУ
    if (pCompressedBuffer4x != nullptr) { delete[] pCompressedBuffer4x; pCompressedBuffer4x = nullptr; }
    pCompressedBuffer4x = new UINT[compW * compH];

    float WEIGHT_R = 0.299f, WEIGHT_G = 0.587f, WEIGHT_B = 0.114f;

    for (UINT mY = 0; mY < compH; mY++)
    {
      for (UINT mX = 0; mX < compW; mX++)
      {
        float GrayMid = 0.0f;
        float GrayMin = 1.0f;
        float GrayMax = 0.0f;
        float GraySqr = 0.0f;

        for (UINT i = 0; i < 8; i++)
        {
          for (UINT j = 0; j < 8; j++)
          {
            UINT pixelX = mX * 8 + j;
            UINT pixelY = mY * 8 + i;

            if (pixelX < imgWidth && pixelY < imgHeight)
            {
              DWORD clr = pRawPixels[pixelY * imgWidth + pixelX];
```

```
                    BYTE r = (clr >> 16) & 0xFF;
                    BYTE g = (clr >> 8) & 0xFF;
                    BYTE b = clr & 0xFF;

                    float GreyValue = (r * WEIGHT_R + g * WEIGHT_G + b * WEIGHT_B) / 255.0f;

                    GrayMid += GreyValue;
                    GraySqr += GreyValue * GreyValue;

                    if (GreyValue > GrayMax) GrayMax = GreyValue;
                    if (GreyValue < GrayMin) GrayMin = GreyValue;
                }
            }
        }

        GrayMid /= 64.0f;
        GraySqr /= 64.0f;
        GraySqr -= GrayMid * GrayMid;
        if (GraySqr > 0.0f) GraySqr = sqrt(GraySqr);

        BYTE finalGray = (BYTE)(GrayMid * 255.0f);
        pCompressedBuffer4x[mY * compW + mX] = (finalGray << 16) | (finalGray << 8) | finalGray;
      }
    }
  }

  /*
  // === ЭТАЛОННЫЙ РАСЧЕТ РАСТЯНУТОГО ФОНА ДЛЯ РЕЖИМА 4 ===
  if (pRawPixels && pCompressedBuffer4x && ::pStretchedBuffer && g_currentImgWidth > 0 &&
g_currentImgHeight > 0)
  {
    UINT compW = g_currentImgWidth / 8;
    UINT compH = g_currentImgHeight / 8;
```

```
for (UINT y = 0; y < g_currentImgHeight; y++)
{
   for (UINT x = 0; x < g_currentImgWidth; x++)
   {
      // Воспроизводим целочисленные сдвиги фаз дефектоскопа 2011 года
      int tex0_x = (int)x - 2;
      int tex0_y = (int)y - 2;

      int tex_x_base = (int)round((float)tex0_x / 4.0f);
      int tex_y_base = (int)round((float)tex0_y / 4.0f);

      int tex_x = tex_x_base / 2;
      int tex_y = tex_y_base / 2;

      int i_x = (tex_x_base % 2) * 4;
      int i_y = (tex_y_base % 2) * 4;

      // Лямбда-функция безопасного чтения макроблоков с ограничением краев кадра
      auto GetBufferGray = [&](int bx, int by) -> float {
         int cl_x = (bx < 0) ? 0 : ((bx >= (int)compW) ? (int)compW - 1 : bx);
         int cl_y = (by < 0) ? 0 : ((by >= (int)compH) ? (int)compH - 1 : by);
         return (float)(pCompressedBuffer4x[cl_y * compW + cl_x] & 0xFF) / 255.0f;
         };

      float k_S = GetBufferGray(tex_x, tex_y);
      float k_LR = GetBufferGray(tex_x + 1, tex_y);
      float k_TB = GetBufferGray(tex_x, tex_y + 1);
      float k_LRTB = GetBufferGray(tex_x + 1, tex_y + 1);

      float k_SS = ((k_S * (8 - i_x) + k_LR * i_x) * (8 - i_y) +
         (k_TB * (8 - i_x) + k_LRTB * i_x) * i_y) / 64.0f;
```

```
            BYTE finalStretchedGray = (BYTE)(k_SS * 255.0f);

            // Записываем результат в наш глобальный полноразмерный массив Режима 4
            ::pStretchedBuffer[y * g_currentImgWidth + x] = (finalStretchedGray << 16) | (finalStretchedGray << 8) | finalStretchedGray;
        }
    }
  }
  */

  // === ЗОЛОТОЕ ПРАВИЛО: ЧИСТО ЗАКРЫВАЕМ ВСЕ РЕСУРСЫ И ПАМЯТЬ БЕЗ УТЕЧЕК! ===
  delete[] pPixelsBuffer;
  pConverter->Release();
  pFrame->Release();
  pDecoder->Release();
  pWICFactory->Release();

  return hr;
}

// === НОВЫЙ КОД: ФУНКЦИЯ ВЫЗОВА СИСТЕМНОГО ПРОВОДНИКА WINDOWS ===
bool OpenFileDialog(HWND hwnd, bool bOpenVideo)
{
  OPENFILENAMEW ofn = {};
  // ==========================================================================
  // >>> ИСПРАВЛЯЕМ СМЕРТЕЛЬНУЮ ЛОВУШКУ ДЛЯ NTDLL.DLL: ДЕЛАЕМ БУФЕР СТАТИЧЕСКИМ
  // ==========================================================================
  static wchar_t szFile[MAX_PATH];
  ZeroMemory(szFile, sizeof(szFile)); // Чисто обнуляем буфер перед каждым вызовом
  // ==========================================================================
  ofn.lStructSize = sizeof(ofn);
  ofn.hwndOwner = hwnd;
```

```
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH; // Жестко задаем максимальный размер

    if (bOpenVideo)
    {
        // Фильтр файлов для видеопотока
        ofn.lpstrFilter = L"Видео файлы (AVI, MP4)\0*.avi;*.mp4\0Все файлы (*.*)\0*.*\0";
        ofn.lpstrTitle = L"Выберите видео поток для дефектоскопа";
    }
    else
    {
        // Фильтр файлов для отдельных изображений
        ofn.lpstrFilter = L"Изображения (BMP, PNG, JPG)\0*.bmp;*.png;*.jpg;*.jpeg\0Все файлы (*.*)\0*.*\0";
        ofn.lpstrTitle = L"Выберите кадр/картинку для анализа";
    }

    ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

    // Распахиваем системное окно Проводника
    if (GetOpenFileNameW(&ofn))
    {
        // Если пользователь выбрал файл и нажал "Открыть" — копируем путь в нашу глобальную переменную
        wcscpy_s(g_szSelectedFilePath, MAX_PATH, ofn.lpstrFile);
        return true;
    }

    return false; // Если пользователь нажал "Отмена"
}
// === КОНЕЦ НОВОГО КОДА ===

// Главная точка входа Windows-приложения
```

```
int WINAPI wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ PWSTR pCmdLine, _In_ int nCmdShow)
{
    // === НОВЫЙ КОД: ИНИЦИАЛИЗАЦИЯ СИСТЕМНОЙ БИБЛИОТЕКИ COM ДЛЯ ДВИЖКА WIC ===
    (void)CoInitializeEx(NULL, COINIT_APARTMENTTHREADED); // Запускает поддержку WIC в Windows

    const wchar_t CLASS_NAME[] = L"MyVideoPlayerWindowClass";
    WNDCLASS wc = { };
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    RegisterClass(&wc);

    hwnd = CreateWindowEx(0, CLASS_NAME, L"Мой Шейдерный Видеоплеер",
        WS_OVERLAPPEDWINDOW | WS_HSCROLL | WS_VSCROLL | WS_CLIPCHILDREN,
        CW_USEDEFAULT, CW_USEDEFAULT,
        800, 600,
        NULL, NULL, hInstance, NULL);

    if (hwnd == NULL) return 0;

    // Инициализируем общие элементы управления Windows
    InitCommonControls();

    // Физически создаем строку статуса, привязанную к главному окну hwnd
    g_hStatusWnd = CreateStatusWindow(
        WS_CHILD | WS_VISIBLE,
        L"HDR Дефектоскоп готов к работе",
        hwnd,                       // Дескриптор вашего окна плеера
        2005
    );
```

```
    if (g_hStatusWnd != NULL && hwnd != NULL)
    {
        // 1. Аппаратно измеряем физическую высоту созданной строки статуса (как в 2004 году!)
        RECT rS = {};
        GetWindowRect(g_hStatusWnd, &rS);
        int statusHeight = rS.bottom - rS.top; // Получили чистую высоту полосы в пикселях

        // 2. Узнаем текущие полные габариты рамы главного окна плеера на экране
        RECT wndRect = {};
        GetWindowRect(hwnd, &wndRect);
        int currentW = wndRect.right - wndRect.left;
        int currentH = wndRect.bottom - wndRect.top;

        // 3. Расширяем внешнюю коробку окна строго на высоту статус-бара!
        // Плеер остаётся в своих прежних размерах, а строка статуса ложится ниже ползунков!
        SetWindowPos(hwnd, NULL, 0, 0, currentW, currentH + statusHeight, SWP_NOMOVE | SWP_NOZORDER | SWP_FRAMECHANGED);
    }

    ShowWindow(hwnd, nCmdShow);

    CreateAppMenu(hwnd); // НОВОЕ: Физически включаем меню на экране сразу после показа окна

    if (FAILED(InitDevice(hwnd)))
    {
        CleanupDevice();
        CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед выходом!
        return 0;
    }

#ifdef my_DRBUG
    CreateDiagnosticWindow(hInstance, hwnd);
```

```
#endif


   MSG msg = { };
   while (WM_QUIT != msg.message)
   {
      // === О Т Л А Д О Ч Н Ы Й   М А Я Ч О К ===
      static bool bLoopStart = true;
      if (bLoopStart) {
         //MessageBoxW(hwnd, L"Маячок Ц-1: Мы успешно вошли в бесконечный цикл обработки сообщений wWinMain!", L"Отладка", MB_OK);
         bLoopStart = false;
      }


      if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
      {
         TranslateMessage(&msg);
         DispatchMessage(&msg);
      }
      else
      {
         Render();
         //RenderSplit(); // Переключили плеер в режим сравнения "До / После"
      }
   }


   CleanupDevice();
   CoUninitialize(); // НОВОЕ: Чисто закрываем и освобождаем ресурсы COM-потока перед выходом!
   return 0;
}


// === ИСТИННАЯ ПРОГРАММНАЯ ОБРАБОТКА ИЗ 2011 ГОДА НА ЦЕНТРАЛЬНОМ ПРОЦЕССОРЕ (CPU) ===
```

```
// === ИСТИННАЯ ПИРАМИДАЛЬНАЯ ФИЛЬТРАЦИЯ 2011 ГОДА НА ПРОЦЕССОРЕ (CPU) ===
HRESULT ApplyCpuFilter(UINT* pSrcPixels, UINT width, UINT height, UINT* pOutPixels)
{
   if (!pSrcPixels || !pOutPixels || width == 0 || height == 0) return E_INVALIDARG;

   // Весовые коэффициенты яркости монохрома из 2011 года
   const float W_R = 0.299f;
   const float W_G = 0.587f;
   const float W_B = 0.114f;

   // ---------------------------------------------------------------------
   // МАССИВ p1: Переводим исходный кадр BGRA в монохромную матрицу яркости
   // ---------------------------------------------------------------------
   float* p1 = new float[width * height];
   for (unsigned int i = 0; i < width * height; i++)
   {
      unsigned int pixel = pSrcPixels[i];
      BYTE b = (pixel & 0x000000FF);
      BYTE g = ((pixel & 0x0000FF00) >> 8);
      BYTE r = ((pixel & 0x00FF0000) >> 16);

      p1[i] = (float)r * W_R + (float)g * W_G + (float)b * W_B;
   }

   // ---------------------------------------------------------------------
   // МАССИВ p2: Первый уровень пирамиды (сжатие блоков 4x4 пикселя)
   // ---------------------------------------------------------------------
   unsigned int w2 = width / 4;
   unsigned int h2 = height / 4;
   if (w2 == 0) w2 = 1;
   if (h2 == 0) h2 = 1;
```

```
float* p2 = new float[w2 * h2];


// Цикл свертки блоков 4x4
for (unsigned int y = 0; y < h2; y++)
{
   for (unsigned int x = 0; x < w2; x++)
   {
      float sum = 0.0f;
      // Сканируем квадрат 4x4 пикселя из исходного массива p1
      for (unsigned int block_y = 0; block_y < 4; block_y++)
      {
         unsigned int src_y = y * 4 + block_y;
         if (src_y >= height) src_y = height - 1; // Защита от выхода за край


         for (unsigned int block_x = 0; block_x < 4; block_x++)
         {
            unsigned int src_x = x * 4 + block_x;
            if (src_x >= width) src_x = width - 1;


            sum += p1[src_y * width + src_x];
         }
      }
      p2[y * w2 + x] = sum / 16.0f; // Среднее арифметическое блока 4x4
   }
}


// ----------------------------------------------------------------------
// МАССИВ p3: Второй уровень пирамиды (укрупнение блоков в 8x8, или 2x2 от p2)
// ----------------------------------------------------------------------
unsigned int w3 = w2 / 2;
unsigned int h3 = h2 / 2;
if (w3 == 0) w3 = 1;
```

```
if (h3 == 0) h3 = 1;

float* p3 = new float[w3 * h3];

// Цикл свертки блоков 2x2 от предыдущего уровня p2
for (unsigned int y = 0; y < h3; y++)
{
   for (unsigned int x = 0; x < w3; x++)
   {
      float sum = 0.0f;
      for (unsigned int block_y = 0; block_y < 2; block_y++)
      {
         unsigned int src_y = y * 2 + block_y;
         if (src_y >= h2) src_y = h2 - 1;

         for (unsigned int block_x = 0; block_x < 2; block_x++)
         {
            unsigned int src_x = x * 2 + block_x;
            if (src_x >= w2) src_x = w2 - 1;

            sum += p2[src_y * w2 + src_x];
         }
      }
      p3[y * w3 + x] = sum / 4.0f; // Среднее арифметическое макроблока
   }
}

// ------------------------------------------------------------------------
// СБОРКА И ФИНАЛЬНЫЙ ВЫВОД (Пока выводим промежуточный массив p1)
// ------------------------------------------------------------------------
   // Цикл высокоточного межполукадрового анализа и интерполяции контуров
for (unsigned int y = 0; y < height; y++)
```

```
  {
    for (unsigned int x = 0; x < width; x++)
    {
      unsigned int idx = y * width + x;
      float src_gray = p1[idx]; // Исходная точка кадра

      // 1. ИНТЕРПОЛЯЦИЯ: Находим координаты текущей точки в укрупненных массивах p2 и p3
      unsigned int x2 = x / 4;
      unsigned int y2 = y / 4;
      if (x2 >= w2) x2 = w2 - 1;
      if (y2 >= h2) y2 = h2 - 1;
      float macro_mid = p2[y2 * w2 + x2]; // Средняя яркость блока 4x4

      unsigned int x3 = x / 8;
      unsigned int y3 = y / 8;
      if (x3 >= w3) x3 = w3 - 1;
      if (y3 >= h3) y3 = h3 - 1;
      float global_mid = p3[y3 * w3 + x3]; // Глобальный фон макроблока 8x8

      // 2. ВЫЧИСЛЕНИЕ ДИСПЕРСИИ И ЛОКАЛЬНОГО КОНТРАСТА
      float delta = src_gray - macro_mid;

      // Внутренний сдвиг локальной разницы (аналог вашего макробарьера p4)
      float local_dispersion = abs(macro_mid - global_mid);

      // Защитный знаменатель против клякс и деления на ноль из 2011 года
      float denom = local_dispersion + (12.75f / 255.0f); // Перевели 12.75 в диапазон [0.0 - 1.0] DirectX
      float local_contrast = delta / denom;

      // Жесткое ограничение диапазона среза, как в HLSL шейдере
      if (local_contrast < -1.5f) local_contrast = -1.5f;
      if (local_contrast > 1.5f)  local_contrast = 1.5f;
```

```
            // 3. ФИНАЛЬНЫЙ СИНТЕЗ И КОРРЕКЦИЯ ДЕФЕКТОСКОПА
            float out_gray = macro_mid + local_contrast * (local_dispersion * 1.8f);

            // Нормализация яркости в стандартный диапазон [0.0, 255.0]
            if (out_gray < 0.0f)   out_gray = 0.0f;
            if (out_gray > 255.0f) out_gray = 255.0f;

            // Честная гамма-коррекция 2.2 силами центрального процессора
            float norm_gray = out_gray / 255.0f;
            float corrected_gray = pow(norm_gray, 1.0f / 2.2f);

            // Проявление скрытых микроконтуров
            corrected_gray = corrected_gray * 1.05f - 0.02f;
            if (corrected_gray < 0.0f) corrected_gray = 0.0f;
            if (corrected_gray > 1.0f) corrected_gray = 1.0f;

            // Собираем готовый пиксель обратно в формат BGRA для вывода шторки
            BYTE final_byte = (BYTE)(corrected_gray * 255.0f);
            pOutPixels[idx] = (0xFF000000) | (final_byte << 16) | (final_byte << 8) | final_byte;
        }
    }


    // Чистое освобождение динамической памяти из ОЗУ компьютера
    delete[] p1;
    delete[] p2;
    delete[] p3;

    return S_OK;
}
```

```
// Главная точка входа Windows-приложения
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
   switch (uMsg)
   {
   case WM_HSCROLL:
   {
      SCROLLINFO si = {};
      si.cbSize = sizeof(SCROLLINFO);
      si.fMask = SIF_ALL;
      GetScrollInfo(hwnd, SB_HORZ, &si);

      int oldPos = si.nPos;
      switch (LOWORD(wParam))
      {
      case SB_LINELEFT:   si.nPos -= 20; break;  // Кликнули на левую стрелочку
      case SB_LINERIGHT:  si.nPos += 20; break;  // Кликнули на правую стрелочку
      case SB_PAGELEFT:   si.nPos -= si.nPage; break;
      case SB_PAGERIGHT:  si.nPos += si.nPage; break;
      case SB_THUMBTRACK: si.nPos = si.nTrackPos; break; // Тащат ползунок мышкой
      }

      // Зажимаем позицию в физические рамки картинки
      if (si.nPos < 0) si.nPos = 0;
      if (si.nPos > (int)(g_currentImgWidth - si.nPage)) si.nPos = (int)(g_currentImgWidth - si.nPage);

      if (si.nPos != oldPos)
      {
         g_scrollX = si.nPos;

         // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
         float denominatorX = (float)(si.nMax - si.nMin);
```

```
        //g_scrollRatioX = (denominatorX > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorX : 0.0f;
        g_scrollRatioX = (float)(si.nPos - si.nMin) / (float)(si.nMax - si.nMin - si.nPage);

        SetScrollInfo(hwnd, SB_HORZ, &si, TRUE);
        Render();
    }
}
return 0;

case WM_VSCROLL:
{
    SCROLLINFO si = {};
    si.cbSize = sizeof(SCROLLINFO);
    si.fMask = SIF_ALL;
    GetScrollInfo(hwnd, SB_VERT, &si);

    int oldPos = si.nPos;
    switch (LOWORD(wParam))
    {
    case SB_LINEUP:    si.nPos -= 20; break;  // Кликнули на верхнюю стрелочку
    case SB_LINEDOWN:  si.nPos += 20; break;  // Кликнули на нижнюю стрелочку
    case SB_PAGEUP:    si.nPos -= si.nPage; break;
    case SB_PAGEDOWN:  si.nPos += si.nPage; break;
    case SB_THUMBTRACK: si.nPos = si.nTrackPos; break; // Тащат ползунок мышкой
    }

    if (si.nPos < 0) si.nPos = 0;
    if (si.nPos > (int)(g_currentImgHeight - si.nPage)) si.nPos = (int)(g_currentImgHeight - si.nPage);

    if (si.nPos != oldPos)
    {
        g_scrollY = si.nPos;
```

```
        // ВАША ФУНДАМЕНТАЛЬНАЯ ФОРМУЛА ДОЛИ:
        float denominatorY = (float)(si.nMax - si.nMin);
        //g_scrollRatioY = (denominatorY > 0.0f) ? (float)(si.nPos - si.nMin) / denominatorY : 0.0f;
        g_scrollRatioY = (float)(si.nPos - si.nMin) / (float)(si.nMax - si.nMin - si.nPage);

        SetScrollInfo(hwnd, SB_VERT, &si, TRUE);
        Render();
    }

}
return 0;

case WM_SIZE:
{
    // Получаем новые физические размеры клиентской области окна из lParam
    g_wndW = (float)LOWORD(lParam);
    g_wndH = (float)HIWORD(lParam);

    // Если DirectX уже инициализирован — обновляем системный SwapChain буфер экрана
    if (g_pSwapChain)
    {
        // Сбрасываем старый RenderTargetView, чтобы Windows разрешила изменить размер буферов
        if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }

        // Раздвигаем внутренний буфер DirectX под новый размер окна
        g_pSwapChain->ResizeBuffers(0, (UINT)g_wndW, (UINT)g_wndH, DXGI_FORMAT_UNKNOWN, 0);

        // Пересоздаем взгляд на экран под новые габариты
        ID3D11Texture2D* pBackBuffer = NULL;
        g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
        if (pBackBuffer) {
```

```
            g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
            pBackBuffer->Release();
        }
    }
}
break;



    // === НОВЫЙ КОД: ОБРАБОТКА НАЖАТИЙ НА ПУНКТЫ МЕНЮ ===
case WM_COMMAND:
    // В wParam система Windows передает ID нажатого пункта меню
    switch (LOWORD(wParam))
    {
    case IDM_VIEW_CPU:
        // 1. Инвертируем глобальный флаг
        g_bUseCPUProcessing = !g_bUseCPUProcessing;

        // 2. Ставим галочку в меню, используя точный идентификатор кнопки
        CheckMenuItem(GetMenu(hwnd), IDM_VIEW_CPU, g_bUseCPUProcessing ? MF_CHECKED :
MF_UNCHECKED);

        // 3. Командуем окну немедленно перерисовать кадр
        InvalidateRect(hwnd, NULL, FALSE);
        return 0;

        // === НАСТОЯЩЕЕ ВНЕДРЕНИЕ: ОБРАБОТКА НАЖАТИЙ НА МАСШТАБЫ ДЕФЕКТОСКОПА ===
    case IDM_ZOOM_FIT: g_bFitToWindow = true;  g_zoomDivider = 1.0f;  goto my_reset_scroll;
    case IDM_ZOOM_1:   g_bFitToWindow = false; g_zoomDivider = 1.0f;  goto my_reset_scroll;
    case IDM_ZOOM_2:   g_bFitToWindow = false; g_zoomDivider = 2.0f;  goto my_reset_scroll;
    case IDM_ZOOM_4:   g_bFitToWindow = false; g_zoomDivider = 4.0f;  goto my_reset_scroll;
    case IDM_ZOOM_8:   g_bFitToWindow = false; g_zoomDivider = 8.0f;  goto my_reset_scroll;
    case IDM_ZOOM_16:  g_bFitToWindow = false; g_zoomDivider = 16.0f; goto my_reset_scroll;
```

```
my_reset_scroll:
{
   // 1. Возвращаем скроллинг в исходный левый верхний угол кадра
   g_scrollX = 0; g_scrollY = 0;
   g_scrollRatioX = 0.0f; g_scrollRatioY = 0.0f;

   // 2. АВТОМАТИЧЕСКАЯ РАССТАНОВКА ГАЛОЧЕК (V) В МЕНЮ ОКНА
   HMENU hViewMenuRef = GetSubMenu(GetMenu(hwnd), 1); // Находим наше меню "Вид" (индекс 1)
   if (hViewMenuRef)
   {
      HMENU hZoomMenuRef = GetSubMenu(hViewMenuRef, 1); // Находим подменю "Масштаб" внутри "Вида"
      if (hZoomMenuRef)
      {
         // Сбрасываем старые птички со всех 6 пунктов
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_FIT, MF_UNCHECKED);
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_1, MF_UNCHECKED);
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_2, MF_UNCHECKED);
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_4, MF_UNCHECKED);
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_8, MF_UNCHECKED);
         CheckMenuItem(hZoomMenuRef, IDM_ZOOM_16, MF_UNCHECKED);

         // Включаем галочку строго на выбранном оператором режиме!
         UINT activeID = IDM_ZOOM_FIT;
         if (!g_bFitToWindow)
         {
            if (g_zoomDivider == 1.0f)  activeID = IDM_ZOOM_1;
            if (g_zoomDivider == 2.0f)  activeID = IDM_ZOOM_2;
            if (g_zoomDivider == 4.0f)  activeID = IDM_ZOOM_4;
            if (g_zoomDivider == 8.0f)  activeID = IDM_ZOOM_8;
            if (g_zoomDivider == 16.0f) activeID = IDM_ZOOM_16;
```

```
            }
            CheckMenuItem(hZoomMenuRef, activeID, MF_CHECKED);
          }
        }

        // 3. СИНХРОНИЗАЦИЯ ПОЛЗУНКОВ ПОД ВЫБРАННЫЙ МАСШТАБ
        UINT currentMaxX = g_bFitToWindow ? (UINT)g_wndW : (UINT)((float)g_currentImgWidth / g_zoomDivider);
        UINT currentMaxY = g_bFitToWindow ? (UINT)g_wndH : (UINT)((float)g_currentImgHeight / g_zoomDivider);

        SCROLLINFO zoomSiX = {};
        zoomSiX.cbSize = sizeof(SCROLLINFO);
        zoomSiX.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
        zoomSiX.nMin = 0;
        zoomSiX.nMax = (currentMaxX > 0) ? currentMaxX : 1;
        zoomSiX.nPage = (UINT)g_wndW;
        zoomSiX.nPos = 0;
        SetScrollInfo(hwnd, SB_HORZ, &zoomSiX, TRUE);

        SCROLLINFO zoomSiY = {};
        zoomSiY.cbSize = sizeof(SCROLLINFO);
        zoomSiY.fMask = SIF_RANGE | SIF_PAGE | SIF_POS;
        zoomSiY.nMin = 0;
        zoomSiY.nMax = (currentMaxY > 0) ? currentMaxY : 1;
        zoomSiY.nPage = (UINT)g_wndH;
        zoomSiY.nPos = 0;
        SetScrollInfo(hwnd, SB_VERT, &zoomSiY, TRUE);

        // 4. Перерисовываем экран
        InvalidateRect(hwnd, NULL, FALSE);
        return 0;
    }
```

```
        case IDM_FILE_EXIT:
            // Если нажали "Выход" — закрываем окно программы
            DestroyWindow(hwnd);
            return 0;

        case IDM_HELP_ABOUT:
            // Если нажали "О программе" — показываем красивое информационное окно
            MessageBoxW(hwnd,
                L"Шейдерный Видеоплеер-Дефектоскоп v2026\n\n"
                L"Реализован пятипроходный вычислительный конвейер.\n"
                L"Алгоритмы фильтрации локального контраста и дисперсии макроблоков 4x4 и 8x8.\n\n"
                L"Разработчик математического ядра: Курчанов А. Ф.",
                L"О программе",
                MB_OK | MB_ICONINFORMATION);
            return 0;

        case IDM_FILE_OPEN_VIDEO:
            // Вызываем Проводник в режиме фильтрации видео (передаем true)
            if (OpenFileDialog(hwnd, true))
            {
                // Если файл выбран успешно — выведем его путь на экран для проверки
                //MessageBoxW(hwnd, g_szSelectedFilePath, L"Видео поток успешно подключен", MB_OK |
MB_ICONINFORMATION);

                // TODO: Здесь на следующем шаге мы запустим инициализацию Media Foundation
            }
            return 0;

        case IDM_FILE_OPEN_IMAGE:
            // 1. Распахиваем Проводник для выбора картинки
            if (OpenFileDialog(hwnd, false))
            {
```

```
            // 2. Если файл выбран — загружаем его пиксели в Point-текстуру через WIC
            if (SUCCEEDED(LoadTextureFromFile(g_szSelectedFilePath)))
            {
                // 3.
                // Принудительно вызываем Render прямо сейчас,
                // чтобы протолкнуть новые SRV-интерфейсы в GPU!
                Render();
                wchar_t statusBuf[256];

                // Подставляем ваши глобальные переменные ширины и высоты кадра плеера
                wsprintf(statusBuf, L"Габариты кадра: %d x %d пикселей -> %d x %d -> %d x %d",
                    g_srcImgWidth, g_srcImgHeight,
                    g_compressedWidth, g_compressedHeight,
                    g_paddedImgWidth, g_paddedImgHeight);

                // Отправляем этот текст в нижний статус-бар!
                SendMessage(g_hStatusWnd, SB_SETTEXT, 0, (LPARAM)statusBuf);
                //RenderSplit(); // Переключили плеер в режим сравнения "До / После"
            }
            else
            {
                MessageBoxW(hwnd, L"Не удалось декодировать файл изображения через WIC.",
L"Ошибка дефектоскопа", MB_OK | MB_ICONERROR);
            }
        }
        return 0;


    case IDM_FILE_SAVE_RESULT:
    case IDM_VIEW_SPLIT:
    case IDM_VIEW_RESULT_ONLY:
        // Для остальных пунктов пока выведем временную заглушку, чтобы видеть отклик
```

```
            MessageBoxW(hwnd, L"Этот блок конвейера находится в режиме подключения данных.",
L"Информация", MB_OK | MB_ICONINFORMATION);
            return 0;
        }
        break;
        // === КОНЕЦ НОВОГО КОДА ===

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
```